

Optimal Charging Protocol for Quantum Many-Body Battery Using Reinforcement Learning

Mohammed Bilal P S

Roll Number: 19080

*A dissertation submitted for the partial fulfillment of BS-MS dual degree in
Physical Science*

Supervisor:

Victor Mukherjee

IISER, Berhampur

Co-supervisor:

Paolo Andrea Erdman

FU Berlin



Indian Institute of Science Education and Research,

Berhampur

April 2024

Declaration

The work presented in this dissertation has been carried out by me under the guidance of Dr. Victor Mukherjee and Dr. Paolo Andrea Erdman at the Indian Institute of Science Education and Research, Berhampur and FU, Berlin.

This work has not been submitted in part or in full for a degree, a diploma, or a fellowship to any other university or institute. Whenever contributions of others are involved, every effort is made to indicate this clearly, with due acknowledgment of collaborative research and discussions. This thesis is a bona fide record of original work done by me, and all sources listed within have been detailed in the bibliography.

Mohammed Bilal P S

Dated: 19/4/2024

In my capacity as the supervisor of the candidate's project work, I certify that the above statements by the candidate are true to the best of my knowledge.


Victor Mukherjee (Supervisor)

Paolo Andrea Erdman (Co-supervisor)

Dated: 19/4/2024

Acknowledgements

I am deeply grateful for the support of many individuals without whom this research work would not have been possible. Firstly, I express my sincere gratitude to Victor and Paolo for their unwavering support throughout this journey. Their encouragement and guidance were invaluable contributions.

I extend my thanks to Bandita and Anna for their expertise and assistance in clarifying doubts, offering valuable advice, and supporting me in various ways. Additionally, I would like to express my appreciation to Sidhu for his helpful attitude, which greatly enhanced the presentation of my research.

I am grateful to my family for keeping me motivated and keeping me on track. Lastly, I want to acknowledge my beloved for standing by me through ups and downs. Their unwavering belief in me has been a source of strength, and I am thankful for their patience and encouragement throughout this process. Thank you all for helping me reach this milestone. Lastly, I want to express gratitude to myself for believing in my abilities, for never giving up, and for striving to do what is right.

Abstract

Energy, being a basic need of day-to-day human life, has led to a quest for better methods of storing and utilizing it. In this study, we formulated the quantum control problem for charging a quantum battery into a machine learning problem and compared the advantages of this method over the greedy protocol. Our study focused on quantum batteries consisting of 2-5 qubits. We also extended the study to include cases with disorder and achieved decent results.

Chapter 1 provides an introduction to the quantum battery, which is further elaborated upon in Chapter 2. Chapter 3 discusses the concept of machine learning, providing a quick overview followed by a detailed explanation of the Soft Actor-Critic method, which was used for the study. Chapter 4 details the formulation of the problem, while Chapter 5 discusses the results.

Contents

Contents	5
1	2
1.1 Introduction	2
2	4
2.1 Quantum Battery	4
2.1.1 Charging Protocol	5
3	8
3.0.1 Overview of ML	8
3.0.2 Reinforcement Learning	10
3.0.3 Illustration:	11
3.0.4 Reinforcement Learning and Figures of Merits	11
3.0.5 Bellman equation:	14
3.0.6 Types of Reinforcement Learning:	15
3.0.7 Q-learning:	15
3.0.8 Deep Q-learning:	16
3.0.9 Actor-Critic method in Reinforcement Learning	18
3.0.10 Policy learning:	23
4	24
4.1 Modeling the Problem for RL	24
4.1.1 Understanding the Environment	25

4.1.2	The Agent	26
4.1.3	Procedure	26
4.2	greedy protocol	28
4.3	sample case of single spin	28
5		30
5.1	Exploring the Hamiltonian of the total system	30
5.1.1	Exploring Different Coupling Configurations in H_0 . .	30
5.1.2	Configurations in Control Field (H_1) combined with the control of $\alpha(t)$	31
5.2	Results	31
5.3	2-Qubit system	33
5.3.1	Case 1:	33
5.3.2	Case 2:	34
5.4	3-Qubit system	37
5.4.1	Case 1:	39
5.4.2	Case 2:	42
5.5	4-Qubit system	42
5.5.1	Case 1:	45
5.5.2	Case 2:	45
5.6	5-Qubit system	47
5.6.1	Case 1:	50
5.6.2	case 2:	50
5.7	conclusion:	52
Bibliography		56

Chapter 1

1.1 Introduction

With the development in the field of quantum information and the technological advancements in realizing them, the idea of a quantum battery [15, 57, 45] is being discussed with high expectations. Energy, being a crucial concept for human needs, necessitates the development of systems to store and reuse it. While conventional batteries are composed of quantum components, their behavior is typically evaluated within the classical framework. This has led to the exploration of utilizing quantum phenomena such as entanglement and coherence for improved battery performance [4, 41, 16, 27, 31, 43].

There is significant interest in various types of quantum batteries. Examples include Dicke batteries [23], spin chain or spin network batteries, and SYK batteries [59], each with specific advantages[64, 10, 28] and high expectations in their respective research areas. The implementation of these ideas in experiments can be classified into two regimens. Implementation can occur in two different temperature regimes: room temperature and ultra-low temperature. Realizations of quantum batteries include operating Dicke batteries in the quantum coherent regime using superconducting materials[12], semiconductor quantum dots[14], NV-centers in diamond[18, 9, 60], neutral atoms[1], Rydberg atoms[1], and trapped ions[13]. These realizations extend beyond these platforms to many others based on quantum technology.

Considering these active research areas, having a proper protocol for charging, which can be applied to the system and accommodates disorder, will be of significant relevance in the experimental realization of these ideas. Our study aims to develop optimal charging strategies for spin-chain quantum batteries. We utilize machine learning to design a protocol that can be trained once and then deployed to charge the quantum battery which have disorders in a specified magnitude. Our focus is on devising protocols that store more energy than greedy protocols over various time intervals and implementing them in systems with disorder. This endeavor contributes to the advancement of quantum technologies. In the coming chapters we will discuss the methods used, a sample case, formulating the problem and then the results obtained.

Chapter 2

2.1 Quantum Battery

The term quantum battery (QB) was introduced by Fannes and Alicki [5]. It can be understood as a set of N identical quantum units or cells in which energy is stored and can be extracted as work [29, 6, 8, 25]. The quantum battery, along with the control parameter $\alpha(t)$, can be defined by the following Hamiltonian:

$$H = H_0 + \alpha(t)H_1 \quad (2.1)$$

Here, H_0 is the Hamiltonian of the battery, H_1 is the charging field, and $\alpha(t)$ is the time-dependent control parameter that allows us to transfer energy to the quantum battery. We consider the boundary condition for $\alpha(t)$. In our case, we choose $\alpha(t)$ to be in the range $[-1, 1]$.

In our study, we considered the spin-chain Hamiltonian of the system as a nearest-neighbor interacting Hamiltonian, whose general form can be represented along with the charging Hamiltonian as:

$$H_0 = \sum_{i=1}^{N-1} J_{i,i+1}^{\delta} \sigma_i^{\delta} \sigma_{i+1}^{\delta}, \quad (2.2)$$

$$H_1 = \sum_{i=1}^N h_i \sigma_i^{\delta'}. \quad (2.3)$$

where N is the number of qubits in the spin chain battery, σ_i^{δ} is the Pauli matrix

acting on the spin at site i along direction $\delta = x, y, z$, and δ' can take any value from x, y, z different from δ . h_i represents local fields acting on the i -th site, and $J_{i,i+1}^\delta$ denotes the coupling constant between the spins at sites i and $i + 1$ for the direction δ .

Our goal is to find the optimal time-dependent control $\alpha(t)$ that maximizes the energy transferred to the quantum battery. We considered two main cases: the site-dependent case and the site-independent case.

In the site-dependent case, the action of the control parameter differs at each site, resulting in a total of N control parameters. In contrast, the site-independent case has only one parameter that is applied uniformly to all sites. The formats for these cases are discussed in the upcoming chapter.

We selected reinforcement learning to aid our study so that the value of $\alpha(t)$ is predicted by the algorithm, thereby giving us the protocol for the system. We chose δ to be in the x direction and δ' to be in the z direction.

2.1.1 Charging Protocol

The charging of a quantum battery can be achieved through two different methods:

1. **Direct Charging Method:** In this approach [11, 17, 46, 58], a suitable interaction Hamiltonian H_1 is switched on for a specific time period (as illustrated in Figure 2.1). The Hamiltonian for the system takes the form

$$H = H_0 + \alpha(t)H_1$$

. Here, the charging process is viewed as the time evolution of the battery's state $|\psi(t)\rangle$ based on H , which depends on the value of $\alpha(t)$. This parameter $\alpha(t)$ acts as an external control parameter over the system.

2. **Charger-Mediated Protocol:** In this method [30, 7, 26], an auxiliary Hamiltonian H_2 (the charger Hamiltonian) is introduced in order to charge the battery (as illustrated in Figure 2.1). The Hamiltonian for the entire

setup and the Hamiltonian for the composite system in the general form can be represented as follows:

$$\begin{aligned} H &= H_0 + H_2 + \alpha(t)H_1 \\ &= H'_0 + \alpha(t)H_1 \end{aligned}$$

where H_0 is the Hamiltonian of the battery, H_1 is the interaction Hamiltonian, and H_2 is the charger Hamiltonian. The process of charging the battery occurs due to an interaction between H_0 and H_2 that happens for a finite time τ . H'_0 is the combined Hamiltonian for the battery charger system. However, this method faces challenges as it exchanges work with the environment through classical control. The control parameter $\alpha(t)$ modulates the interaction between the charger and the battery Hamiltonian[7]. If $[H_0, H_1] \neq 0$, then the final energy attained by the battery can have the energy gained by the modulation of the control parameter, hence the charge won't be purely due to the charger Hamiltonian[30].

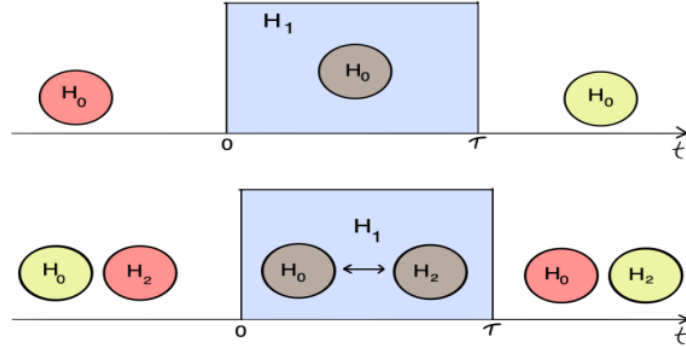


Figure 2.1: Illustration of different charging methods. The first figure shows the direct protocol, and the second figure shows the mediated charging protocol.

Since we are considering the value of $\alpha(t)$ for each time step Δt as per the protocol, we can say that α can be understood as a step function, which is defined in the range for time $t = [0, \tau]$ with a value γ such that it obeys the boundary condition $\gamma \in [-1, 1]$ and zero otherwise ($t \notin [0, \tau]$). This idea can be experimentally maintained with a setup of an external magnetic field.

The total energy ($E(\tau)$) of the battery at the end of the charging protocol ($t = \tau$) can be expressed as:

$$E(\tau) = \langle \psi(\tau) | H_0 | \psi(\tau) \rangle \quad (2.4)$$

We can define the optimal charging time $\tilde{\tau}$ as:

$$E(\tilde{\tau}) = \max_{\tau > 0} E(\tau) \quad (2.5)$$

Thus, we can see that the value of $\alpha(t)$ - the control parameter in each step - is very important since it helps the system undergo time evolution and meet the desired requirements. It is also important to note that the minimum time for this time evolution is bounded based on the quantum speed limit (QSL) [34, 22], which states that if we have $|\psi_1\rangle$ and $|\psi_2\rangle$ as two pure states, then the minimum time τ' required to unitarily evolve between them is bound as:

$$\tau' \geq \tau_{QSL} = \frac{\arccos |\langle \psi_1 | \psi_2 \rangle|}{\min\{\langle E \rangle, \langle \sigma_E \rangle\}} \quad (2.6)$$

where $\langle E \rangle$ and $\langle \sigma_E \rangle$ are the time-averaged expectation value and standard deviation of the total Hamiltonian $H = H_0 + \alpha(t)H_1$.

It is important to understand that the QSL gives the lower bound for the time evolution of a given H , hence the choice of $\alpha(t)$ has an impact on the minimal time as well. This question of finding the best $\alpha(t)$ for minimizing τ' is known as the quantum brachistochrone, which is a difficult problem to solve, and hence we have solutions for only a few cases.

Chapter 3

A sneak peek into Machine Learning

3.0.1 Overview of ML

Observing the surroundings and learning to predict patterns is a natural inclination for humans. With the development of computational methods, the idea of using computational devices to learn as humans do is flourishing. Machine learning, or machine learning methods[39, 47] in general, leverage this aspiration and execute them with the aid of statistics, probability, and mathematical concepts like function approximation, linear algebra, etc. These ideas in general can be classified as follows

- **Supervised Learning:** In supervised learning [54], the computer is provided with labeled data to learn from. This means the computer is given examples where it knows the correct answer. For instance, imagine we want to teach a computer to approximate a function $F(x)$, which is unknown to us, but we are provided with its value on a set of points. More specifically, assume we are given a dataset of points $D = \{x_i, y_i\}_{i=1, \dots, N}$, such that $y_i = F(x_i)$. The goal in supervised learning is to learn an approximation of the unknown function $F(x)$ using the dataset D . This is typically done considering some function approximator $f_\theta(x)$, where θ is a set of tunable parameters. $f_\theta(x)$ could be, for instance, a neural network, and θ would represent the weights and biases of the network. The values of θ that produce the best function approximator $f_\theta(x)$ are found

minimizing a loss function $\mathcal{L}(\theta)$ that represents the error between the predicted output $f_{\theta}(x_i)$ and the actual output y_i . One common loss function is the mean squared error between the prediction and the labels:

$$\mathcal{L}(\theta) = \sum_{i=1}^N (y_i - f_{\theta}(x_i))^2.$$

- **Unsupervised Learning:** In unsupervised learning [32], the computer must uncover patterns and structures within data without the aid of labeled examples. Consider a dataset without labels, where our goal is to classify data points based solely on their inherent properties. For instance, let's take a collection of integers ranging from 1 to 100. Utilizing the K-means[61] clustering algorithm, we aim to partition these numbers into two distinct groups: one for even numbers and another for odd numbers. Initially, the algorithm randomly selects two cluster centroids and assigns each data point to the nearest centroid. Through iterative refinement, the centroids are adjusted to minimize the distance between data points and their respective centroids. Eventually, the algorithm converges, resulting in two clusters: one containing even numbers and the other odd numbers. This example showcases how unsupervised learning techniques, like K-means clustering, enable computers to autonomously classify data based on inherent properties, offering valuable insights without the need for labeled data.
- **Reinforcement Learning:** Another approach to machine learning involves allowing a computer agent to learn by directly interacting with an environment. In reinforcement learning (RL) [55, 49, 50], the computer agent learns to make decisions and take actions by trial and error, receiving feedback from the environment in the form of rewards or penalties. The primary objective in RL is for the agent to learn how to optimally choose actions to maximize the cumulative reward it receives over time. Overall, reinforcement learning provides a powerful framework for train-

ing computer agents to make autonomous decisions and learn complex behaviors through interaction with their environment. We will discuss more in the coming session.

3.0.2 Reinforcement Learning

Reinforcement learning (RL). is a method in machine learning where an agent learns to make decisions by interacting with an environment as illustrated by the Figure3.2. The core concept of RL revolves around the interaction between an agent and its environment, where the agent learns from the consequences of its actions. Through this process, the agent strives to achieve a goal, often defined in terms of maximizing cumulative rewards.

At the heart of RL lies the Markov Decision Process (MDP)[56] concept, which formally defines the environment and the agent's interactions within it. An MDP is characterized by a tuple (S, A, P, R, γ) , where:

- S is the set of all possible states in the environment.
- A is the set of all possible actions that the agent can take.
- $P(s' | s, a)$ defines the probability of transitioning to state s' given that the current state is s and action a is taken.
- $R(s, a, s')$ specifies the immediate reward obtained upon transitioning from state s to state s' by taking action a .
- γ is a discount factor, typically between 0 and 1, determining the importance of future rewards relative to immediate rewards.

MDP[56] provides a mathematical framework for modelling sequential decision-making under uncertainty. It captures the dynamics of the agent's environment and enables the agent to learn an optimal policy by interacting with it over time. In RL, the agent aims to find a policy that maximizes the expected cumulative reward over an infinite time horizon, considering both immediate rewards and future consequences.

3.0.3 Illustration:

For example, let's take a look at the CartPole Environment provided by OpenAI Gym. In this scenario, there's a pole attached to a cart, and the job of the agent is to keep the pole balanced by moving the cart to the left or right. Each time the agent takes an action, like pushing the cart, the environment gives feedback in the form of a reward r and a state S .

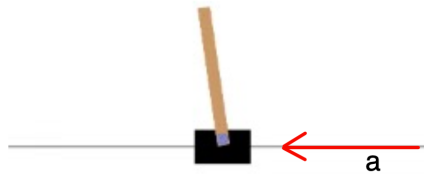


Figure 3.1: illustration of cart pole environment of Open AI gym environment, the figure shows the cartpole and action a applied to the cart

The goal here is to keep the pole from falling over, as shown in Figure 3.1. To do this, the agent needs to know about the state of the cartpole, which includes things like the cart's velocity, position, the pole's angular velocity, and the pole's angle. When the agent successfully keeps the pole upright, it gets a reward of +1. However, there are boundaries to consider, like the cart's position and the pole's angle, which can't go beyond certain values.

During training, the agent learns which actions to take, like pushing left or right, to keep the pole balanced. If the boundaries are reached, the training starts over, and this process is repeated many times to learn the best strategy.

In reinforcement learning (RL), the agent's learning journey involves trying out different actions and learning from the environment's feedback. Through this trial-and-error process, the agent gradually improves its decision-making skills and works towards achieving its goal.

3.0.4 Reinforcement Learning and Figures of Merits

To understand the ideas behind reinforcement learning, let's break it down into its core components and equations:

In reinforcement learning, the **agent** serves as the learner or decision-maker interacting with the environment. It functions as a mapping from the current state S_t and reward $r(t)$ to the value of A_t . The **environment**, exemplified by

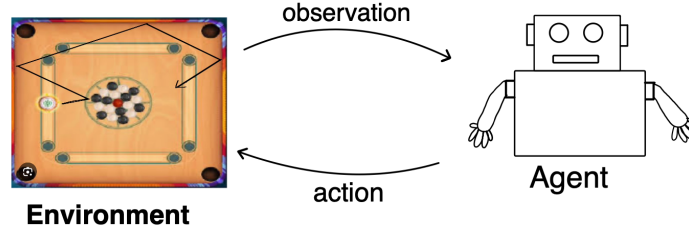


Figure 3.2: schematic of a RL algorithm, we can see that for a given action a the environment returns the observation i.e the new state and the reward

the classic cart-pole problem, constitutes the external system with which the agent engages. Defined by the state S_t containing variables—cart position (x), cart velocity ($\dot{x}$), pole angle (θ), and pole angular velocity ($\dot{\theta}$)—the environment dynamics are subject to the agent’s actions A_t , typically pushing the cart left or right. Governed by physics equations, the system’s state evolves over time in response to these actions. The selection of the action is based on the **policy** $\pi(a|s)$, which dictates the agent’s strategy or rule for action selection, mapping states to actions. The change in the state of the cart pole is valued as good or bad based on the **reward** (r), a scalar feedback signal reflecting the quality of the action within the current state. In our example, a reward of +1 is bestowed for maintaining the pole upright. The sequence of interactions between the agent and environment, originating from an initial state and concluding upon reaching a terminal state, is defined as an **episode**, which constitutes a singular iteration of the learning process.

The important aspect of the whole process is to know the quality of the state reached by the action of the agent on the environment. It is quantified by the **value function** (V), which is expressed as the expected cumulative reward an agent can achieve starting from a particular state and following a specific policy. Mathematically, it is defined as the expected return, denoted by $E[G_t|S_t = s]$, where G_t represents the cumulative reward from time step t onward, and $S_t = s$

denotes that the agent is in state s at time step t . The value function V is then expressed as:

$$V(s) = E[G_t | S_t = s]$$

Since the Value function provides the quality of a state, it is also important to know the quality of the action to be taken for a given state, which is quantified by the **Q-value** ($Q(s, a)$)[65], which represents the expected cumulative reward of taking action a in a particular state s and following a specific policy thereafter. It is defined as:

$$Q(s, a) = E[G_t | S_t = s, A_t = a]$$

Where G_t represents the cumulative reward from time step t onward, given that the agent is in state s and takes action a at time step t , which for a given reward $r(t)$ can be represented as follows:

$$G(t_i) = \sum_{k=0}^{\infty} \gamma^k r(t_{i+k+1})$$

This equation gives the sum of rewards from time t onwards, discounted by a factor $\gamma \in [0, 1]$. The Q-value[65] provides a more detailed estimate of potential rewards compared to the value function by considering both the state and the action taken. These functions play crucial roles in guiding the agent's decision-making process in reinforcement learning tasks.

In short, we can summarize the fundamental aspect of reinforcement learning as the notion of state-action-reward (SAR) pairs. At each time step, the agent observes the current state of the environment, selects an action based on its policy, and receives a corresponding reward or penalty. By associating actions with their resulting rewards, the agent learns which actions lead to desirable outcomes and which should be avoided. One popular RL algorithm that exemplifies this concept is SARSA (State-Action-Reward-State-Action)[66, 3], where the agent updates its policy by considering both the current state-action pair and

the subsequent state-action pair, along with the rewards obtained. Through repeated interactions with the environment, the agent refines its decision-making process, gradually improving its performance in pursuit of its objectives. In the next section we will delve into the equation that underpins RL, the Bellman equation [2, 48, 35]

3.0.5 Bellman equation:

The Bellman equation is a fundamental concept in dynamic programming and reinforcement learning. It represents the relationship between the value function at a certain point in time and the value function at later points in time, taking into account the immediate reward and the value of future decisions. This can be mathematically expressed by the equation

$$V(s) = \max_a \left[\sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V(s')) \right] \quad (3.1)$$

$V(s)$ is the value of state s . a is an action taken in state s . $P(s'|s, a)$ is the transition probability from state s to state s' under action a . $R(s, a, s')$ is the immediate reward received after transitioning from state s to state s' under action a . γ is the discount factor that determines the importance of future rewards.

The Bellman equation for the Q-value, also known as the action-value function, expresses the value of taking a particular action in a given state, considering the immediate reward and the discounted value of future states. It can be written as:

$$Q(s, a) = \sum_{s'} P(s'|s, a) \left(R(s, a, s') + \gamma \max_{a'} Q(s', a') \right) \quad (3.2)$$

Where $Q(s, a)$ is the Q-value, representing the expected cumulative reward of taking action a in state s and following the optimal policy thereafter. a' represents all possible actions in state s' .

3.0.6 Types of Reinforcement Learning:

Reinforcement learning (RL)[55, 49, 50] algorithms can be categorized based on how they learn and update their policies. Two primary categories are on-policy and off-policy methods.

1. On-policy RL:

On-policy RL[63] methods learn the value or policy function from the data collected by following the current policy. They optimize the policy that is currently being used for action selection. One example of an on-policy RL algorithm is the SARSA algorithm.

Example: SARSA (State-Action-Reward-State-Action) algorithm updates its Q-values based on the current policy and then follows that policy for action selection and further updates.

2. Off-policy RL:

Off-policy RL[33] methods learn the value or policy function from data generated by following a different policy. They aim to optimize a policy different from the one used to generate the data. One popular off-policy RL algorithm is Q-learning.

Example: Q-learning updates its Q-values based on the maximum Q-value of the next state, regardless of the policy being followed.

Both types of RL algorithms have their advantages and are used in various applications depending on the specific requirements of the task and the available data.

3.0.7 Q-learning:

Q-learning[20] is a model-free reinforcement learning algorithm that learns the optimal action-value function $Q^*(s, a)$. Q learning is focused on working for

discrete action space(eg: the case of cart pole we mentioned earlier).The idea is to iteratively update the Q-values based on the Bellman optimality equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where:

- $Q(s, a)$ is the Q-value for taking action a in state s .
- r is the immediate reward received after taking action a .
- s' is the next state after taking action a .
- α is the learning rate.
- γ is the discount factor.

The agent explores the environment by selecting actions based on an exploration strategy (e.g., ϵ -greedy). Q-learning updates the Q-values based on the observed rewards and transitions, gradually converging to the optimal action-value function.

3.0.8 Deep Q-learning:

Deep Q-learning[24, 40] extends Q-learning by using a deep neural network to approximate the Q-values. Neural networks are powerful mathematical models capable of approximating complex functions. They consist of interconnected layers of neurons, where each neuron applies a weighted sum of inputs followed by a non-linear activation function. The network, commonly referred to as the Q-network takes the state as input and outputs Q-values for all actions. The Q-network is trained to minimize the temporal difference (TD) error:

$$\mathcal{L} = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta') - Q(s, a; \theta) \right)^2 \right]$$

where:

- θ are the parameters of the Q-network.
- θ' are the parameters of a target network, periodically updated from θ .

TD learning updates the value function based on the difference between the current state's estimated value and the next state's estimated value, along with the immediate reward received upon transitioning between states. Training is done by sampling experiences from a replay buffer and updating the Q-network parameters using gradient descent to minimize the TD error loss.

Deep Q-learning with experience replay and target networks improves stability and convergence, enabling effective learning in complex environments with high-dimensional state spaces.

training procedure Deep Q-Learning (DQN) is a variant of Q-Learning that uses deep neural networks to approximate the Q-function. Here's a detailed explanation of the training process in DQN:

1. **Initialization:** Initialize two neural network(NN) for the Q-value- $Q - \theta(s, a)$, where θ are the training parameters of the NN and initialize the replay buffer to store experiences.
2. **Exploration vs. Exploitation:** Choose an action using an ϵ -greedy policy, where with probability ϵ , a random action is chosen (exploration), and with probability $1 - \epsilon$, the action with the highest Q-value according to the current neural network is chosen (exploitation)[42, 62, 19].
3. **Experience Replay:** Interact with the environment by executing actions and observing rewards and next states. Store each experience tuple (s, a, r, s') in the replay buffer[52].
4. **Sample Batches:** Sample a mini-batch of experiences from the replay buffer to update the neural network parameters.
5. **Target Q-Value:** Compute the target Q-value for each experience in the mini-batch which is kept as a fixed value function while gradient descent. we

can define it's value using the Bellman equation as :

$$Q_{\theta'}^{\text{target}}(s, a) = r + \gamma \max_{a'} Q_{\theta}(s', a')$$

6. **Loss Calculation:** Compute the loss between the predicted Q-values and the target Q-values using the Mean Squared Error (MSE)[21] loss function:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (Q_{\theta'}^{\text{target}}(s_i, a_i) - Q_{\theta}(s_i, a_i))^2$$

7. **Gradient Descent:** Update the Q-network $Q_{\theta}(s, a)$ weights to minimize the loss using gradient descent.
8. **Update Target Network:** Periodically update a target network with the weights of the current Q-network to stabilize training.
9. **Repeat:** Repeat steps 2-8 for multiple episodes until convergence.

Both Q-learning and Deep Q-learning are off-policy algorithms, meaning they can learn from experiences generated by any policy while still learning the optimal policy.

3.0.9 Actor-Critic method in Reinforcement Learning

Introduction to Actor-Critic Networks in Reinforcement Learning

Deep Q-Learning (DQN)[24, 40] has been a groundbreaking technique in reinforcement learning (RL), allowing agents to learn optimal policies in complex environments. However, DQN suffers from limitations such as high variance and instability during training. Also it's important to understand that DQN are modeled for discrete actions. These issues stem from the use of a single neural network to both estimate action values and select actions, leading to difficulty in effectively updating the network parameters. To address the limitations of DQN

and improve the efficiency and stability of RL algorithms, an upgrade is proposed: the adoption of Actor-Critic networks. Actor-Critic networks combine the benefits of both value-based and policy-based methods by simultaneously learning a value function (Critic) and a policy (Actor).

In reinforcement learning, Actor-Critic methods are a class of algorithms that combine the ideas of both policy-based and value-based methods. They consist of two main components:

- **Actor Network:** This network represents the policy and is responsible for selecting actions based on the current state. The actor network outputs a probability distribution over actions given a state.
- **Critic Network:** This network evaluates the actions chosen by the actor. It provides feedback to the actor by estimating the value function or the action-values (Q-values). The critic network helps in learning the value of taking certain actions in certain states.

The collaboration between the Actor and Critic networks allows for more stable and efficient learning in RL tasks. The Actor selects actions based on the policy learned by the Critic, while the Critic provides feedback on the effectiveness of the actions taken. This symbiotic relationship between the Actor and Critic networks forms the foundation of Actor-Critic algorithms in RL. The Actor-Critic architecture merges the benefits of value-based and policy-based approaches, fostering stable and efficient learning. In this framework, actor and critic networks collaborate: the actor network devises a policy mapping states to actions, while the critic network evaluates these actions.

The actor network's policy is denoted by $\pi_\theta(a|s)$, parameterized by θ . It is trained to maximize the expected return:

$$\mathcal{J}_{\text{actor}}(\theta) = \mathbb{E}_{s \sim \rho^\pi} [\mathbb{E}_{a \sim \pi_\theta} [Q^\pi(s, a)]]$$

where ρ^π denotes the state distribution under policy π .

The critic network's action-value function is denoted by $Q_\phi(s, a)$, parameterized by ϕ . It is trained to minimize the Bellman error:

$$\mathcal{L}_{\text{critic}}(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(r + \gamma Q_{\phi'}(s', \pi_{\theta'}(s')) - Q_{\phi}(s, a))^2 \right]$$

where $\mathcal{D}$ represents the experience replay buffer and ϕ' and θ' are the target network parameters.

Soft Actor-Critic (SAC) Method

Soft Actor-Critic (SAC)[38, 36, 37] is an off-policy actor-critic deep RL algorithm that maximizes the expected return while also maximizing the entropy of the policy. The idea of entropy regularization introduced to control the exploration exploitation balance. The entropy of a stochastic policy is defined as the measure of uncertainty or randomness in the distribution of actions. For a continuous action space, the entropy $H(\pi)$ of a policy π can be calculated as:

$$H(\pi) = - \int_{\mathcal{A}} \pi(a|s) \log \pi(a|s) da$$

where $\mathcal{A}$ is the action space, $\pi(a|s)$ is the probability density function (PDF) of selecting action a given state s , and da denotes the differential element in the action space.

Entropy Regularization

Entropy-regularized reinforcement learning[67, 51] introduces entropy bonuses to the agent's policy, providing a trade-off between exploration and exploitation. In this setting, the agent receives a bonus reward at each time step proportional to the entropy of the policy. The objective of the agent is to maximize the expected sum of discounted rewards along with the entropy bonus. Formally, the optimal policy π^* is obtained by maximizing the following objective:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot|s_t))) \right]$$

where $\alpha > 0$ is the trade-off coefficient, $R(s_t, a_t, s_{t+1})$ represents the immediate reward, γ is the discount factor, and $H(\pi(\cdot|s_t))$ denotes the entropy of

the policy at time step t . This formulation transforms the reinforcement learning problem into maximizing the expected sum of rewards and entropy bonuses over trajectories. We consider an infinite-horizon discounted setting throughout this discussion.

The value functions in this entropy-regularized setting are modified to incorporate the entropy bonuses. The state-value function $V^\pi(s)$ now includes the sum of rewards and entropy bonuses from every timestep, and can be expressed as

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot|s_t)) \right) \middle| s_0 = s \right]$$

While the action-value function $Q^\pi(s, a)$ includes the sum of rewards and entropy bonuses from every timestep except the first, and is expressed as:

$$Q^\pi(s, a) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) + \alpha \sum_{t=1}^{\infty} \gamma^t H(\pi(\cdot|s_t)) \middle| s_0 = s, a_0 = a \right]$$

Furthermore, $V^\pi(s)$ and $Q^\pi(s, a)$ are interconnected through the Bellman equation, which can be understood by the relation:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi} [Q^\pi(s, a)] + \alpha H(\pi(\cdot|s)) \\ Q^\pi(s, a) &= \mathbb{E}_{s' \sim P, a' \sim \pi} [R(s, a, s') + \gamma(Q^\pi(s', a') + \alpha H(\pi(\cdot|s')))] \\ &= \mathbb{E}_{s' \sim P} [R(s, a, s') + \gamma V^\pi(s')]. \end{aligned}$$

which ensures consistency between the state and action values. This framework allows for effective exploration in reinforcement learning tasks by balancing the agent's desire for high rewards with its need for exploration and uncertainty.

Entropy regularization encourages exploration by discouraging the policy from becoming overly deterministic. It leads to a more robust and generalized policy, capable of handling various environments and tasks. By balancing the trade-off between exploration and exploitation, SAC promotes both effective

exploration of the environment and efficient exploitation of learned policies.

Lose function for Actor and Critic Networks

SAC[38, 36, 37] concurrently learns a policy π_θ and two Q-functions Q_{ϕ_1} and Q_{ϕ_2} . There are two variants of SAC that are currently standard: one that uses a fixed entropy regularization coefficient α , and another that enforces an entropy constraint by varying α over the course of training.

Let's consider the equation for $Q^\pi(s, a)$ and express it with the substitution of the equation for the entropy:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_{s' \sim P, a' \sim \pi} [R(s, a, s') + \gamma(Q^\pi(s', a') + \alpha H(\pi(\cdot|s')))] \\ &= \mathbb{E}_{s' \sim P, a' \sim \pi} [R(s, a, s') + \gamma(Q^\pi(s', a') - \alpha \log \pi(a'|s'))]. \end{aligned}$$

the notation for the next action is switched to $\tilde{a}'$, instead of a' , to highlight that the next actions have to be sampled fresh from the policy.

One can notice that we have expressed it as an expectation over future states and actions whose information is obtained from the replay buffer and the current policy, respectively. Since we have our equation as an expectation, we can approximate it to:

$$Q^\pi(s, a) \approx r + \gamma (Q^\pi(s', \tilde{a}') - \alpha \log \pi(\tilde{a}'|s')), \quad \tilde{a}' \sim \pi(\cdot|s').$$

SAC sets up the Mean Squared Bellman Error (MSBE) loss for each Q-function. Hence, we can express the loss functions for the Q-networks in SAC as:

$$\mathcal{L}(\phi_i, \mathcal{D}) = \mathbb{E}_{(s, a, r, s', d) \sim \mathcal{D}} \left[(Q_{\phi_i}(s, a) - y(r, s', d))^2 \right],$$

where the target is given by

$$y(r, s', d) = r + \gamma(1-d) \left(\min_{j=1,2} Q_{\phi_{\text{targ},j}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot|s').$$

3.0.10 Policy learning:

The policy is to be learned such that it helps to maximize the expected future return plus expected future entropy. That is, it should maximize $V^\pi(s)$, which can be understood as:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi}[Q^\pi(s, a)] + \alpha H(\pi(\cdot|s)) \\ &= \mathbb{E}_{a \sim \pi}[Q^\pi(s, a) - \alpha \log \pi(a|s)]. \end{aligned}$$

Handling Continuous Action Spaces

SAC[38, 36, 37] is particularly well-suited for environments with continuous action spaces. Let's consider an environment where the action is continuous and bounded between a and b . In order to achieve this, we keep the output of our policy network as two numbers that are treated as the mean(μ) and standard deviation(σ). Once we have them, we sample a value from a Gaussian with the mean and variance we obtained. Then, the action A is obtained from the equation

$$\begin{aligned} A &= a + \frac{b-a}{2} (1 + \tanh \epsilon) \\ \epsilon &\sim \mathcal{N}(\mu, \sigma^2) \end{aligned}$$

where ϵ is sampled from a Gaussian distribution with μ and σ^2 as the mean and variance, respectively.

Handling continuous action spaces in SAC allows for precise control of the system, enabling it to perform delicate manipulations and achieve desired outcomes with accuracy. SAC's ability to handle continuous action spaces makes it suitable for a wide range of real-world applications in quantum control and optimization tasks.

Chapter 4

Modeling the Environment

4.1 Modeling the Problem for RL

In this section, we discuss how we formulate the problem of optimizing the energy of a quantum battery as a reinforcement learning problem. We then describe how we can create a reinforcement learning environment representing the charging of a quantum battery and examine a sample problem to gain a better understanding of the issue.

First, we discretize the time interval $[0, \tau]$ into n timesteps of duration $\Delta t = \tau/n$. Let us denote the discrete time-steps as $t_i = i \times \Delta t$, for $i = 0, 1, \dots, n$. The goal of RL is to learn how to choose all actions α_i , such that we maximize the value of the cumulative reward G , which is the sum of all rewards obtained during an episode (during the charging). To apply RL to our charging problem, we must specify the state of the environment. As the state of the environment, we simply choose the wave-function $|\psi(t_i)\rangle$, the action α_i , and the reward r_i .

More specifically, we can fix some basis of the Hilbert space of the battery, project $|\psi(t)\rangle$ onto this basis, and stack the real and imaginary parts of these coefficients into a 1D vector. Additionally, we must also include the current time t_i into the state of the environment.

As action α_i , we choose the value of the control $\alpha(t_i)$ that we keep constant

for a short time Δt in the time interval $[t_i, t_{i+1}]$. So essentially

$$\alpha_i = \alpha(t) \quad \text{for } t \in [t_i, t_{i+1}].$$

This way, the agent will discover a piece-wise constant protocol $\alpha(t)$ where the value is constant in each interval $[t_i, t_{i+1}]$.

As reward r_{i+1} , we choose the variation of energy of the quantum battery, i.e.

$$r_{i+1} = E(t_{i+1}) - E(t_i).$$

The reason is the following. As mentioned above, the goal of RL is to maximize the return G . Using the equations mentioned above, we can understand that

$$\begin{aligned} G &= r_1 + r_2 + \dots + r_n = \\ &= E(t_1) - E(t_0) + E(t_2) - E(t_1) + \dots + E(t_n) - E(t_{n-1}) = \\ &= E(\tau) - E(0). \end{aligned} \quad (4.1)$$

Since the initial energy $E(0)$ is fixed, we see that this choice of the reward will maximize the final energy $E(\tau)$, which is precisely our goal.

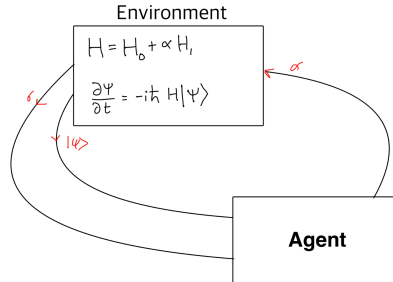


Figure 4.1: Schematic of a RL algorithm. r denotes the charge of the battery, $|\psi(t)\rangle$ represent the new state, α represent the scalar control parameter.

4.1.1 Understanding the Environment

Now we will look into the components of the environment and how it facilitates the dynamics of the system. It takes in a state $|\psi(t_i)\rangle$ and performs the time

evolution based on the Schrödinger equation. It returns the next state $|\psi(t_{i+1})\rangle$, calculates the expectation value/energy $E(t_{i+1}) = \langle \psi(t_{i+1}) | H_0 | \psi(t_{i+1}) \rangle$, produces the reward $r(t_{i+1}) = E(t_{i+1}) - E(t_i)$ that is to be given to the agent (refer Figure 4.1), and keeps track of the time constraints (i.e., $t_0 \leq t \leq t_n$), where the boundary conditions are assigned as $t_0 = 0$ and $t_n = \tau$.

4.1.2 The Agent

The agent intakes the current state $|\psi(t_i)\rangle$ and the reward $r(t_i)$, then returns the value of the control parameter $\alpha(t_i)$ to the environment. This process repeats until $t_i = t_n$.

4.1.3 Procedure

1. Using the number of particles N provided by the user, the Hamiltonian for the system H_0 and the control field H_1 are created according to the equations 2.1. The boundary conditions are applied to the variables as follows: $t_0 = 0$ and $\alpha(t_0) = 0$.
2. The ground state $|g.s\rangle$ is found for H_0 . In case of degeneracy, the ground state is randomly selected among them and the initial state $|\psi(t = 0)\rangle = |g.s\rangle$. Additionally, the eigenvalues are computed, and the minimum value ($E_{\min}$) and the maximum value ($E_{\max}$) are noted. The ground-states and the eigen values are found with the help of Qutip[44] method `eigenstates()`
3. The Hamiltonian for time evolution is constructed based on the equation $H = H_0 + \alpha(t_i)H_1$.
4. The new state is defined as $|\psi(t_{i+1})\rangle = \hat{U}(0, \Delta t)|\psi(t_i)\rangle$, where $\hat{U}(0, \Delta t)$ is a unitary operator that evolves the state using the Schrödinger equation from $t = 0$ to $t = 10^{-2}$ secs. In our study, this is calculated using the

function `sesolve()` in Qutip, a quantum toolbox. the expectation value $E(t_i)$ is computed along with the reward $r(t_i)$.

5. Using the value of $r(t_i)$ and $|\psi(t_i)\rangle$ the value of $\alpha(t_i)$ is obtained from the agent.
6. Steps 3 to 5 are repeated until $t_i = t_n$ i.e . $t_i = \tau$

In the initial stages of learning, the values of $\alpha(t_i)$ are determined according to the exploitation vs exploration method. The time evolution process is carried out for a specific time step (in our case, each step is split as $\Delta t = 10^{-2}$ sec). The process is completed when $t_i = \tau$, and the machine learning algorithm learns using this information and updates the policy parallelly to optimize for maximum rewards.

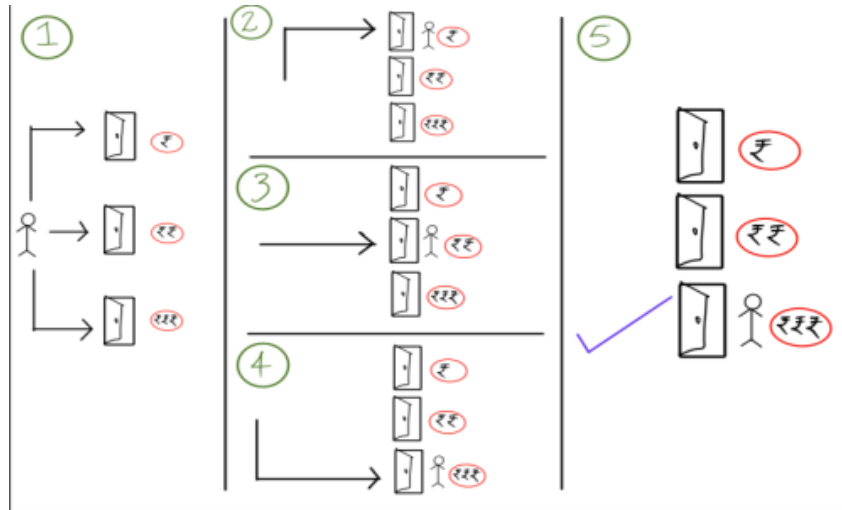


Figure 4.2: Illustration of a simple case to understand the greedy protocol. Here, Bob is asked to choose one from any of the 3 different doors in front of him. He will check each of them sequentially, and from his observation, he finds the best door that has a better reward and chooses that as his choice. In short, he visited each door to check the reward and chose the one with the better reward.

4.2 greedy protocol

Since our objective is to find a optimal stratgy that can perferom better,we consider the idea of greedy protocl to be the becnhmark fro this comparison the greedy protocol is employed to determine the best value for the control $\alpha(t_i)$ between the discrete action values from $\{-1, 0, 1\}$ for a given time t_i and $|\psi(t_i)\rangle$. These values are selected as $\alpha(t_{i+1}) = \text{argmax}\{E_{-1}(t_{i+1}), E_0(t_{i+1}), E_1(t_{i+1})\}$, where $E_{-1}(t_{i+1})$, $E_0(t_{i+1})$, and $E_1(t_{i+1})$ represent the expectation values expressed as $E_j(t_{i+1}) = \langle \psi_j(t_{i+1}) | H_0 | \psi_j(t_{i+1}) \rangle$, $j \in \{-1, 0, 1\}$, and $|\psi_j(t_{i+1})\rangle$ is the time evolved state of $|\psi(t)\rangle$ with respect to H , with the action $\alpha(t_i) = j$. This selection process is iterated until time $t = \tau$. the discussed concept can also be understood using the illustraion 4.2

4.3 sample case of single spin

In this part, we will take an example to learn and understand the ideas of a quantum battery. Let's define the Hamiltonian of the system and field as

$$\begin{aligned} H_1 &= \sigma_z \\ H_0 &= \sigma_x \\ H &= H_0 + \alpha(t)H_1 \end{aligned}$$

Let's consider the states $|\uparrow\rangle$ and $|\downarrow\rangle$ such that:

$$\begin{aligned} \sigma_x |\uparrow\rangle &= |\uparrow\rangle \\ \sigma_x |\downarrow\rangle &= -|\downarrow\rangle \\ \sigma_z |\downarrow\rangle &= |\uparrow\rangle \\ \sigma_z |\uparrow\rangle &= |\downarrow\rangle \end{aligned}$$

considering the hamiltoninan of the battery H_0 we have the ground state as $|\downarrow\rangle$ which we will be considering as the intital state .our objective is to find the

control parameter $\alpha(t)$ that can time-evolve the state into the excited state i.e. $|\uparrow\rangle$. Let's set the boundary conditions for this situation as $\alpha(t) \in [-1, 1]$ for all $t \in [0, \tau]$ and $\alpha(t=0) = 0$. The initial state $\tau = 15$ sec.

To solve this situation we model our environment using the ideas mentioned in the above section. Once the environment is modelled we train it using the RL algorithm.

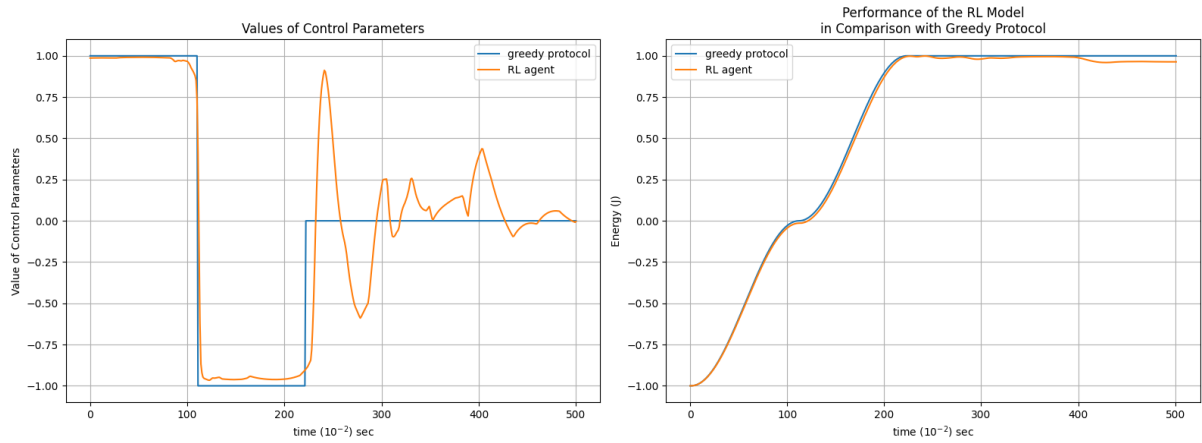


Figure 4.3: values of the control parameter obtained by the RL agent and Comparison of the model and greedy protocol for a single qubit quantum battery for $\tau = 1$ sec

We can see that our model has learned the control and is performing similarly to the greedy protocol, as illustrated by Figure 4.3. One can notice that the policy learned by the RL agent is similar to the greedy protocol, and these results highlight the fact that the RL agent is able to learn about the environment perfectly. It's also interesting to note that our results align with the idea that optimal charging of a quantum battery can be obtained by the bang-bang modulation of the external field Hamiltonian H_1 , i.e., by taking the extreme values of the control parameter [53].

Chapter 5

5.1 Exploring the Hamiltonian of the total system

considering the given Hamiltonian of the total system

$$H = \sum_{i=1}^{N-1} J_{i,i+1} \sigma_i^x \sigma_{i+1}^x + \alpha(t) \sum_{i=1}^N h_i \sigma_i^z$$

one can identify two different possibilities considering H_0 and two different situations for H_1 .

5.1.1 Exploring Different Coupling Configurations in H_0

We have our Hamiltonian for the system as $H_0 = \sum_{i=0}^{N-1} J_{i,i+1} \sigma_i^x \sigma_{i+1}^x$. The possible configurations are:

- **Case 1:** $J_{i,j} = \lambda, \forall \{i, j\}$. In our study, we took $\lambda = 1$.
- **Case 2:** $J_{i,j} = \lambda_{i,j}$, where $\lambda_i \sim \mathcal{N}(\mu, \sigma^2)$ and λ_i and scaled in to the values within $[\delta, 1]$, such that the λ_i will be bounded within δ and 1. In our study, we took $\mu = 0$, $\sigma = 1$ and different cases for δ takes value from $\{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$. This is done by scaling the sampled values into the limit for each cases

5.1.2 Configurations in Control Field (H_1) combined with the control of $\alpha(t)$

Let's define the field Hamiltonian along with the action of the control on each site as

$$\begin{aligned} H_{comb} &= \alpha(t)H_1 \\ &= \sum_{i=1}^N h'_i \sigma_i^z \end{aligned}$$

where $\{i\}$ represents the sites and $h'_i = h_i \times \alpha_i(t)$. The possible configurations are:

- **Site Independent:** $h'_i = h_i \alpha_i(t), \forall \{i\}$, where $\alpha_i(t)$ is the control parameter given by the agent at each time t and has the same value for all sites (i.e., $\alpha_i(t) = \text{constant}, \forall \{i\}$ at a given time t) and is bounded within $[-1, 1]$. In our study, $h_i = 1$, hence we can say that h'_i is also bounded within $[-1, 1]$.
- **Site Dependent:** $h'_i = h_i \alpha_i(t), \forall \{i\}$, where $\alpha_i(t)$ is the control parameter given by the agent at each time t and can have different values for each site (i). In our study, $h_i = 1$, hence we can say that h'_i is also bounded within $[-1, 1]$.

From here onward, we will consider the total Hamiltonian of the system with the action of the control as:

$$\begin{aligned} H &= H_0 + H_{comb} \\ &= \sum_{i=0}^{N-1} J_{i,i+1} \sigma_i^x \sigma_{i+1}^x + \sum_{i=0}^N h'_i \sigma_i^z \end{aligned}$$

5.2 Results

In this section, we will go through the results obtained by the RL algorithm and compare them with the greedy protocol(for case 1 only) . Let's define the

following variables:

$$l = e_{\max} - e_{\min}$$

were $e_{\max}, e_{\min}$ are the minimum and maximum eigen values of the system in consideration.

lets define the charge (in percentage) at time t_i as:

$$C(t_i) = \frac{E(t_i) - e_{\min}}{l} \times 100\%$$

let's consider the states $|\uparrow\rangle$ and $|\downarrow\rangle$ such that:

$$\begin{aligned}\sigma_x |\uparrow\rangle &= |\uparrow\rangle \\ \sigma_x |\downarrow\rangle &= -|\downarrow\rangle \\ \sigma_z |\downarrow\rangle &= |\uparrow\rangle \\ \sigma_z |\uparrow\rangle &= |\downarrow\rangle\end{aligned}$$

In our study, we investigated two cases:

Case 1:

- We considered $N = \{2, 3, 4, 5\}$ particles and $\tau = \{1, 2, 4, 6, 8, 10\}$ seconds, with $h'_i \in [-1, 1]$ for site-dependent and site-independent cases.
- The choice of τ was based on the idea that allowing the system to charge for a longer time would lead to maximum charging. Intermediate values were chosen to evaluate the policy found by the agent and compare it with a long-term charging protocol, which might differ.
- We aimed to compare the performance at different time intervals and compare them with a greedy charging protocol.

Case 2:

- We varied the values of δ from $\{0.95, 0.90, 0.85, 0.80, 0.75, 0.70, 0.65, 0.60\}$ and considered $\tau = 1\text{sec}$ and $N = \{2, 3, 4, 5\}$.

- The choice of δ and τ was to observe the protocol's performance under increasing disorder, for short periods.

5.3 2-Qubit system

In this section, we will deal with a 2 qubit quantum battery with the Hamiltonian of the system as H_0 and the charging field along with the action control parameter H_{comb} defined as:

$$H_0 = J_{1,2} \sigma_1^x \otimes \sigma_2^x$$

$$H_{comb} = h'_1 \sigma_1^z + h'_2 \sigma_2^z$$

For the given Hamiltonian H_0 , we have the highest and lowest eigenvalues and their corresponding eigenvectors as (C_{norm} is chosen so that the state in consideration is normalized):

$$e_{\max} = 1, \quad |\psi_1\rangle \in C_{\text{norm}} \times \{|\uparrow\uparrow\rangle, |\downarrow\downarrow\rangle\}$$

$$e_{\min} = -1, \quad |\psi_2\rangle \in C_{\text{norm}} \times \{|\downarrow\uparrow\rangle, |\uparrow\downarrow\rangle\}$$

The $|\psi_1\rangle$ and $|\psi_2\rangle$ are the possible configurations for the excited state and for the ground state of system. The value of l according to the definition can be obtained as $l = 2$. Also, the value for $C(t)$ for some t can be defined as

$$C(t) = \frac{E(t) + 1}{2} \times 100\%$$

5.3.1 Case 1:

We set $J_{1,2} = 1$, thus the Hamiltonian of the system can be represented as:

$$H_0 = \sigma_1^x \otimes \sigma_2^x$$

. Since we have degeneracy, the initial state will be chosen randomly among the obtained eigenvectors corresponding to the minimum eigenvalue ($e_{\min}$), i.e., $|\psi_{\text{initial}}\rangle = |\psi_2\rangle$. In the context of our study, both of our models performed remarkably well, as illustrated by the performance curve in Figure 5.1, with $C_{\text{model}}(\tau) \geq C_{\text{greedy}}(\tau)$ and achieving $C_{\text{model}}(\tau) > 95\%$ for all $\tau \in \{2, 4, 6, 8, 10\}$. The detailed comparison of the greedy protocol and the model's performance is mentioned in Tables 5.1 and 5.2. From Figure 5.2, we can see that the greedy protocol takes a very long time to reach 100% compared to the models (both site-independent and dependent).

Table 5.1: Performance comparison of both the models with the greedy protocol for a 2 qubit quantum battery. Here SD represents the site-dependent case, SID represents the site-independent case

$\tau(\text{sec})$	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	0.61	0.6	0.62	0.59
2	0.63	0.63	1.0	0.97
4	0.66	0.66	1.0	1.0
6	0.69	0.69	1.0	0.99
8	0.73	0.72	1.0	1.0
10	0.75	0.75	1.0	1.0

Table 5.2: Charge (in percentage) obtained $C(\tau)$ for each protocol in Table 5.1

τ	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	80.5	80.0	81.0	79.5
2	80.5	81.5	100.0	98.5
4	83.0	83.0	100.0	100.0
6	84.5	84.5	100.0	99.5
8	86.5	86.0	100.0	100.0
10	87.5	87.5	100.0	100.0

5.3.2 Case 2:

We set the value $J_{i,j}$ to vary in each episode. We can see that the disordered case is well handled by the model. It is also noted that as the value of δ in-

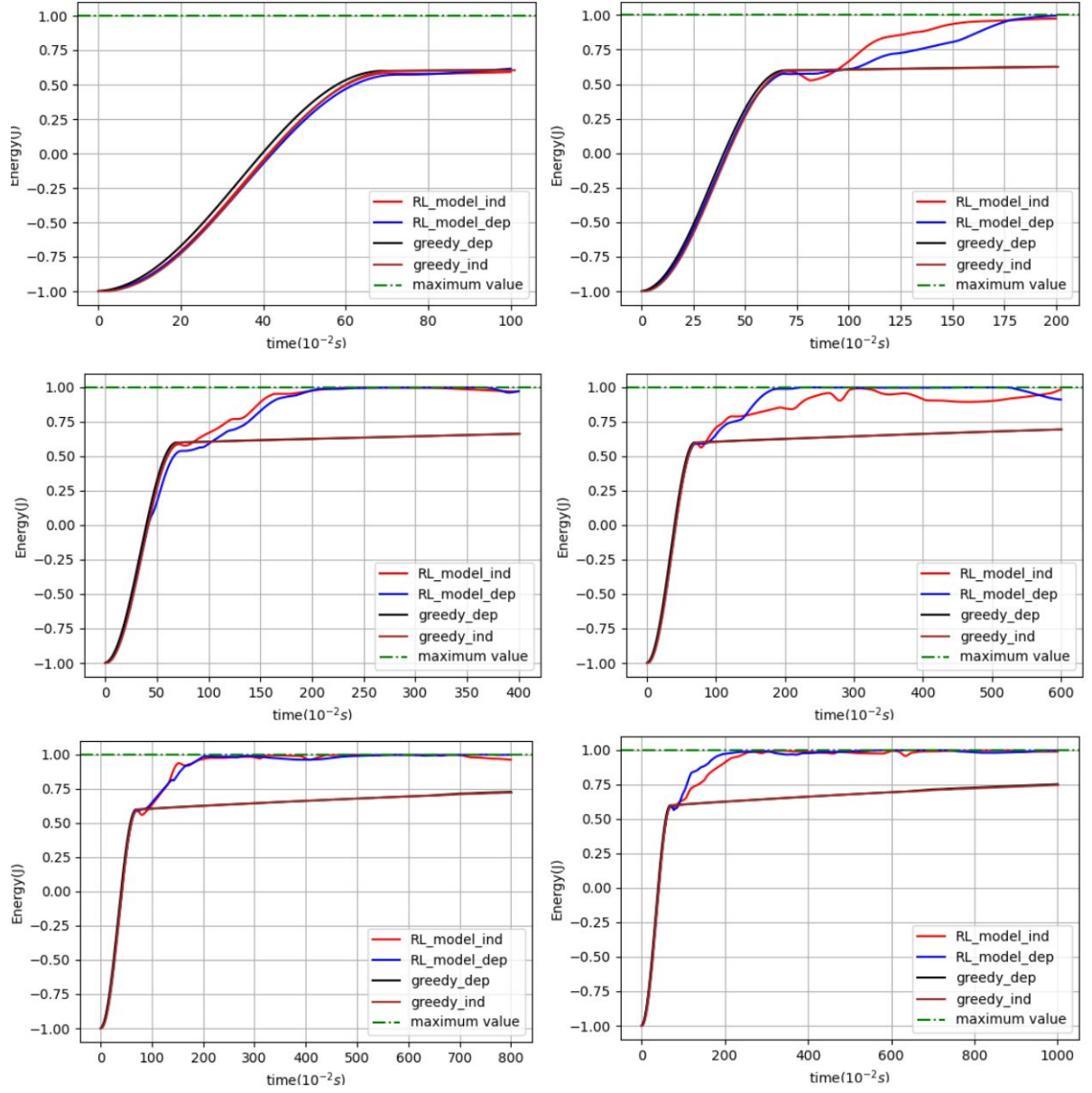


Figure 5.1: Performance of the model in comparison with the greedy protocol for a 2 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$

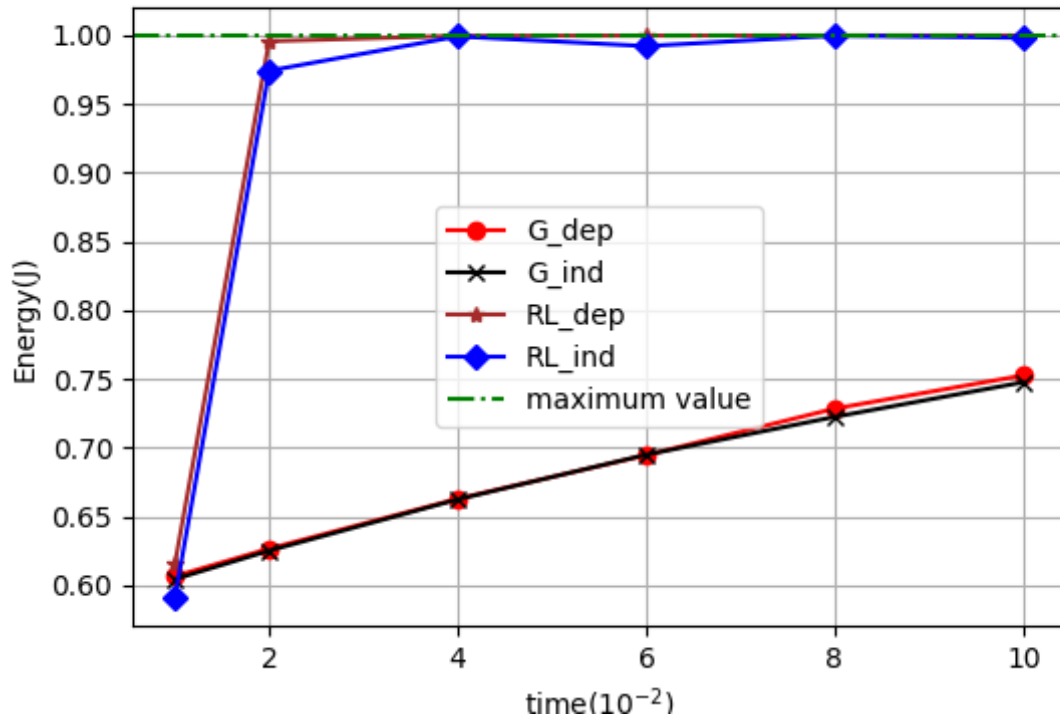


Figure 5.2: Comparison of the model and greedy protocol for a 2 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$

creases, i.e., the disorder decreases, the system attains better results. This can be understood from Figure 5.4 and 5.3. For a 2-qubit quantum battery, the energy obtained seems to be similar without much difference when considering the site-independent and site-dependent cases. The battery charge $C(1)$ is greater than 75% for all cases for $\tau = 1$, which when compared with the normal case is less as mentioned in 5.3,5.2.

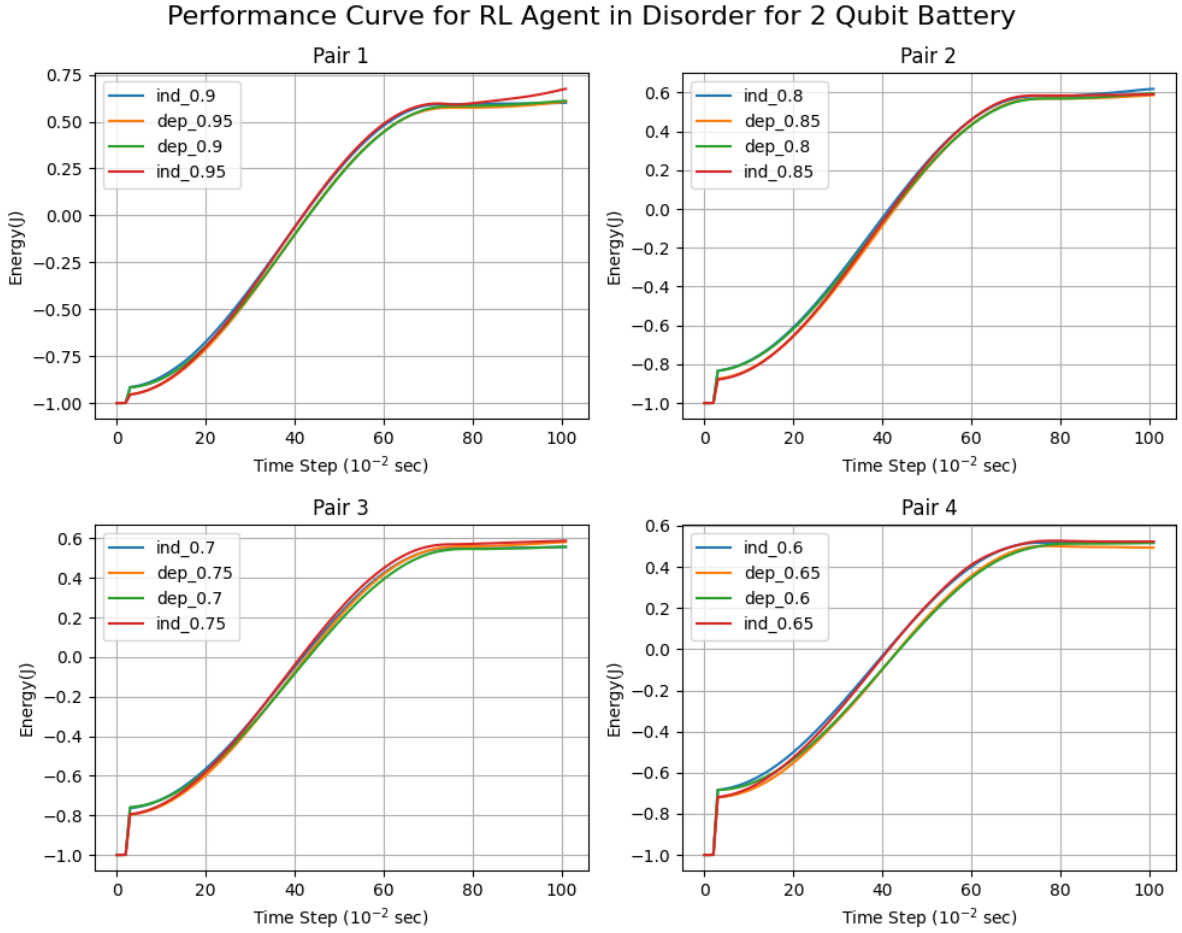


Figure 5.3: Performance of the model in a disordered system for a 2-qubit quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

5.4 3-Qubit system

In this section, we will deal with a 3 qubit quantum battery with the Hamiltonian of the system as H_0 and the charging field along with the action of the control

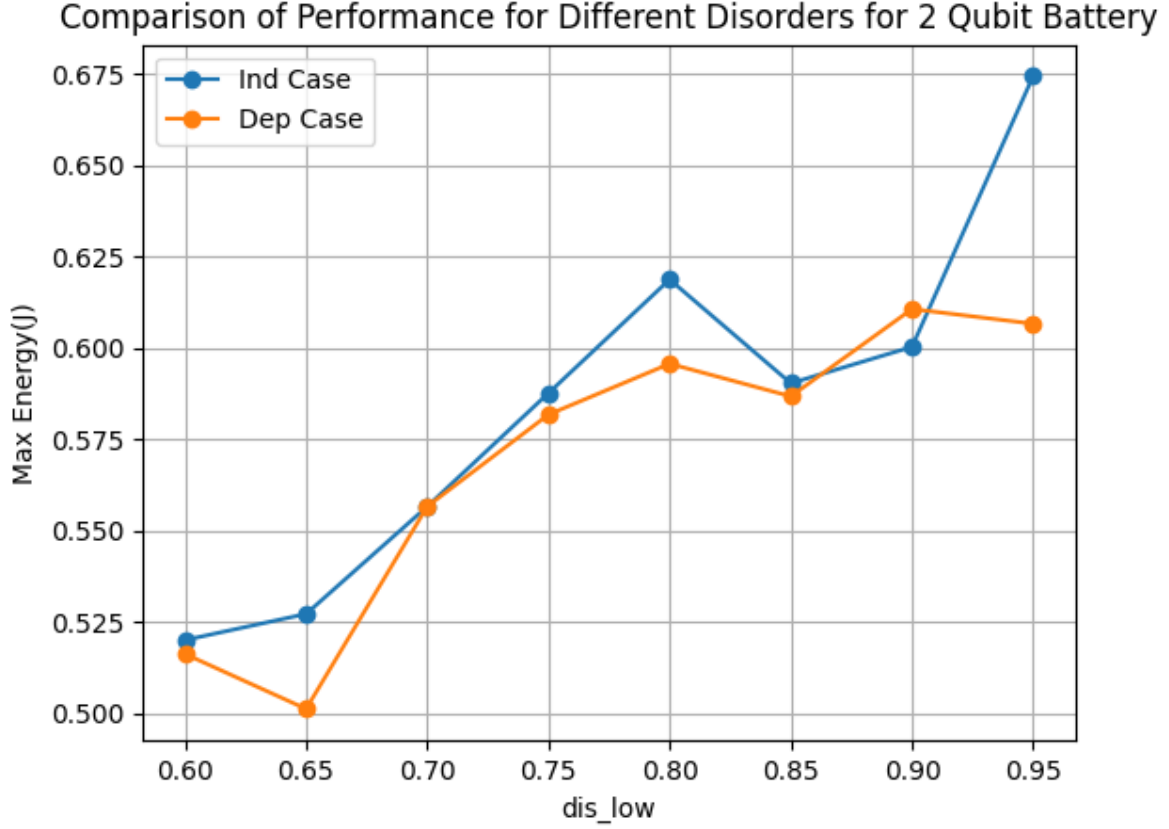


Figure 5.4: Maximum energy obtained by the model in disorder for a 2-quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

Table 5.3: Performance comparison of both the models in a disordered 2 qubit quantum battery. Here SD represents the site-dependent case, SID represents the site-independent case, δ is the lower bound for the disorder as mentioned in the above sections, $C_{SID}(\tau)$ and $C_{SD}(\tau)$ represent the percentage of charge attained by the battery at time $t = 1$ sec for a site independent and site dependent case respectively.

δ	SID	$C_{SID}(\tau)$	SD	$C_{SD}(\tau)$
0.95	0.6744	83.72	0.6066	80.33
0.90	0.6003	80.01	0.6106	80.53
0.85	0.5903	79.52	0.5867	79.34
0.80	0.6187	80.94	0.5957	79.78
0.75	0.5876	79.38	0.5817	79.09
0.70	0.5565	77.83	0.5566	77.83
0.65	0.5271	76.36	0.5011	75.06
0.60	0.5200	76.00	0.5162	75.81

parameter as H_{comb} defined as:

$$\begin{aligned} H_0 &= J_{1,2}\sigma_1^x \otimes \sigma_2^x + J_{2,3}\sigma_2^x \otimes \sigma_3^x \\ H_{\text{comb}} &= h'_1\sigma_1^z + h'_2\sigma_2^z + h'_3\sigma_3^z \end{aligned}$$

For the given Hamiltonian, we have the highest and lowest eigenvalues and their corresponding eigenvectors as:

$$\begin{aligned} e_{\max} &= 2, \quad |\psi_1\rangle \in \{|\uparrow\uparrow\uparrow\rangle, |\downarrow\downarrow\downarrow\rangle\} \\ e_{\min} &= -2, \quad |\psi_2\rangle \in \{|\uparrow\downarrow\uparrow\rangle, |\downarrow\uparrow\downarrow\rangle\} \end{aligned}$$

The $|\psi_1\rangle$ and $|\psi_2\rangle$ are the possible configurations for the excited state and for the ground state of the system. The value of l according to the definition can be obtained as $l = 4$. Also, the value for $C(t)$ for some t can be defined as:

$$C(t) = \frac{E(t) + 2}{4} \times 100\%$$

5.4.1 Case 1:

We set $J_{i,i+1} = 1, \forall i \in \{1, 2\}$, thus the Hamiltonian of the system can be represented as:

$$H_0 = \sigma_1^x \otimes \sigma_2^x + \sigma_2^x \otimes \sigma_3^x$$

Since we have degeneracy, the initial state will be chosen randomly among the obtained eigenvectors corresponding to the minimum eigenvalue ($e_{\min}$), i.e., $|\psi_{\text{initial}}\rangle = |\psi_2\rangle$. Our model's performance seems to vary over time, as illustrated by the performance curve in Figure 5.5, with $C_{\text{model}}(\tau) \geq C_{\text{greedy}}(\tau)$ and achieving $C_{\text{model}}(\tau) > 98\%$ for all $\tau \in \{2, 4, 6, 8, 10\}$. The detailed results are mentioned in Tables 5.4 and 5.5. The model's (site-independent) performance is very inconsistent for $0 < t < \tau$, for $\tau \in \{8, 10\}$, even though it catches up as $t \rightarrow \tau$. This shows the need for fine-tuning the architecture for the RL agent.

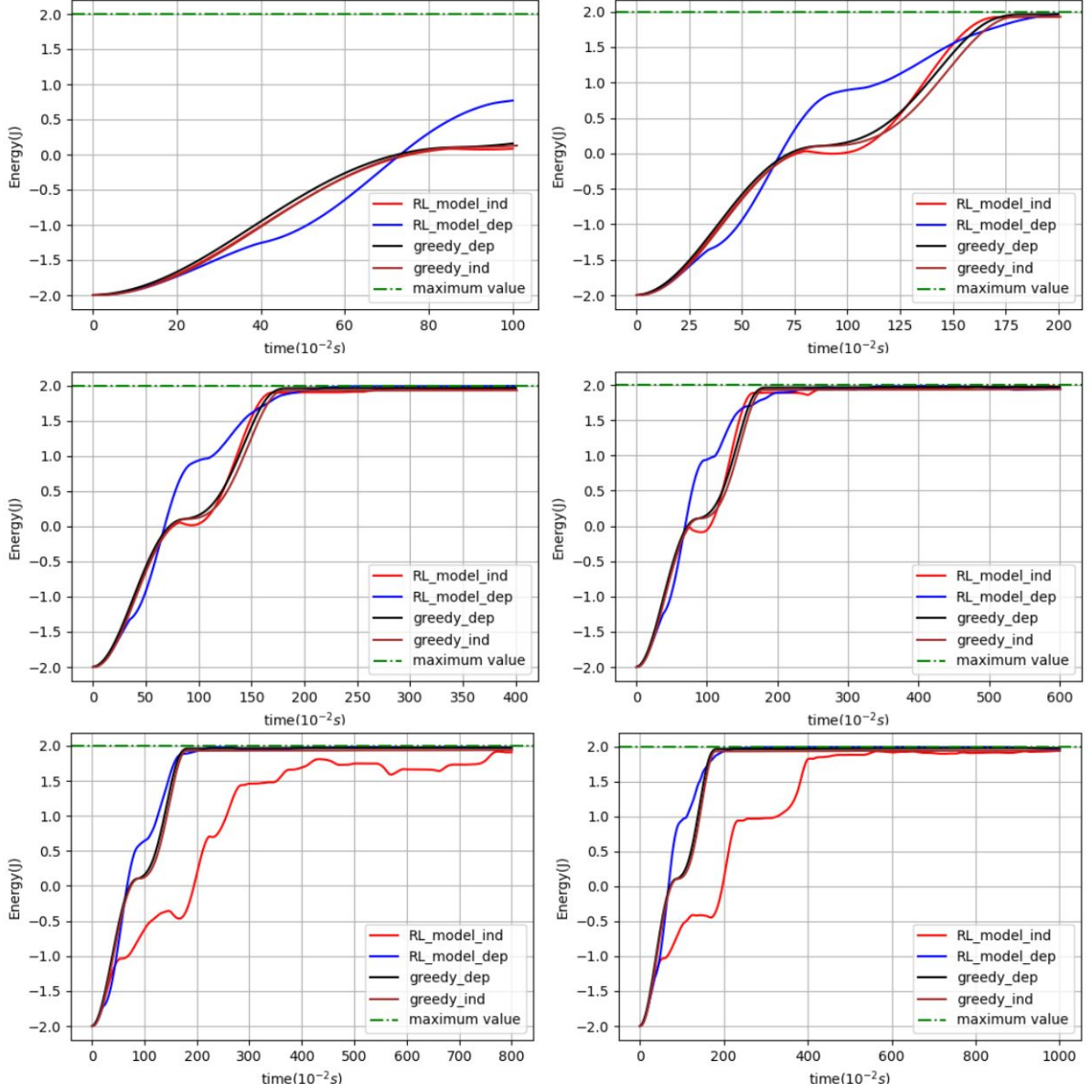


Figure 5.5: Performance of the model compared with the greedy protocol for a 3 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$

Table 5.4: Performance comparison of both the models with the greedy protocol for a 3-qubit quantum battery. Here SD represents the site-dependent case, SID represents the site-independent case

τ (sec)	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	0.16	0.13	0.77	0.08
2	1.96	1.93	1.97	1.93
4	1.97	1.93	2.00	1.96
6	1.97	1.94	1.99	1.99
8	1.97	1.94	1.99	1.92
10	1.98	1.94	1.99	1.94

Table 5.5: Charge (in percentage) obtained $C(\tau)$ for each protocol in Table 5.4

τ (sec)	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	54.0	53.25	69.25	52.0
2	99.0	98.25	99.25	98.25
4	99.25	98.25	100.0	99.0
6	99.25	98.5	99.75	99.75
8	99.25	98.5	99.75	98.0
10	99.5	98.5	99.75	98.5

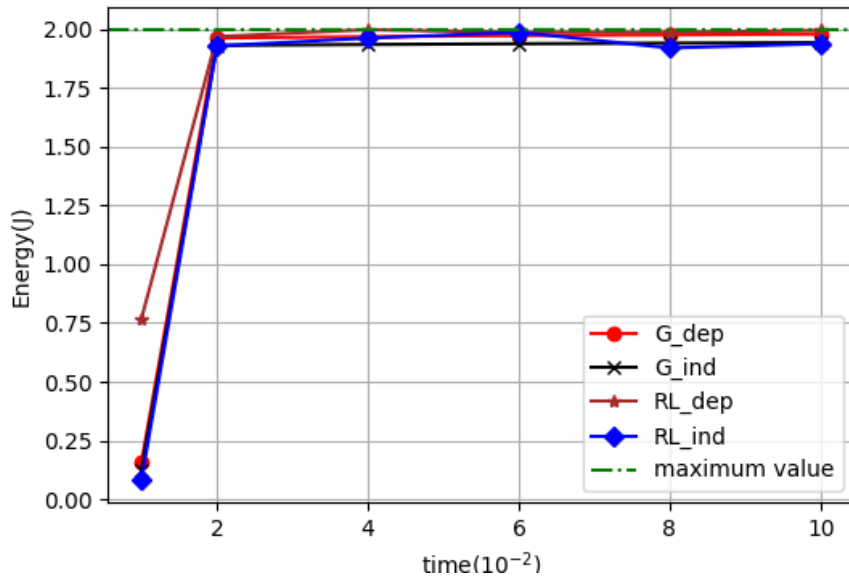


Figure 5.6: Comparison of energy obtained by a 3 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$, here the x-axis is in seconds

5.4.2 Case 2:

We set the value $J_{i,j}$ to vary in each episode. We can see that the disordered case is well handled by the model. It is also noted that as the value of δ increases, i.e., the disorder decreases, the system attains better results. This can be understood from Figure 5.8 and 5.7. For a 3-qubit quantum battery, the energy obtained seems better for the site-dependent case over the independent case. The battery attains near to 54% in the case of site independent and a value greater than 67% in the site dependent case as illustrated by Table 5.5. Also, the performance of the agents in both the disordered case and normal cases are comparable, and there is not a huge margin in difference in terms of the attained charge percentage; this can be understood from Table 5.6 and 5.5.

Table 5.6: Performance comparison of both the models in a disordered 3 qubit quantum battery. Here SD represents the site-dependent case, SID represents the site-independent case, δ is the lower bound for the disorder as mentioned in the above sections, $C_{SID}(\tau)$ and $C_{SD}(\tau)$ represent the percentage of charge attained by the battery at time $t = 1$ sec for a site-independent and site-dependent case respectively.

δ	SID	$C_{SID}(\tau)$	SD	$C_{SD}(\tau)$
0.95	0.1623	54.06	0.8226	70.57
0.90	0.1363	53.41	0.8420	71.05
0.85	0.1508	53.77	0.8292	70.73
0.80	0.1493	53.73	0.7525	68.81
0.75	0.1600	54.00	0.7896	69.74
0.70	0.1506	53.77	0.7636	69.09
0.65	0.1576	53.94	0.7325	68.31
0.60	0.1589	53.97	0.6957	67.39

5.5 4-Qubit system

In this section, we will deal with a 4 qubit quantum battery with the Hamiltonian of the system as H_0 and the charging field along with the action of $\alpha(t)$ as H_{comb} defined as:

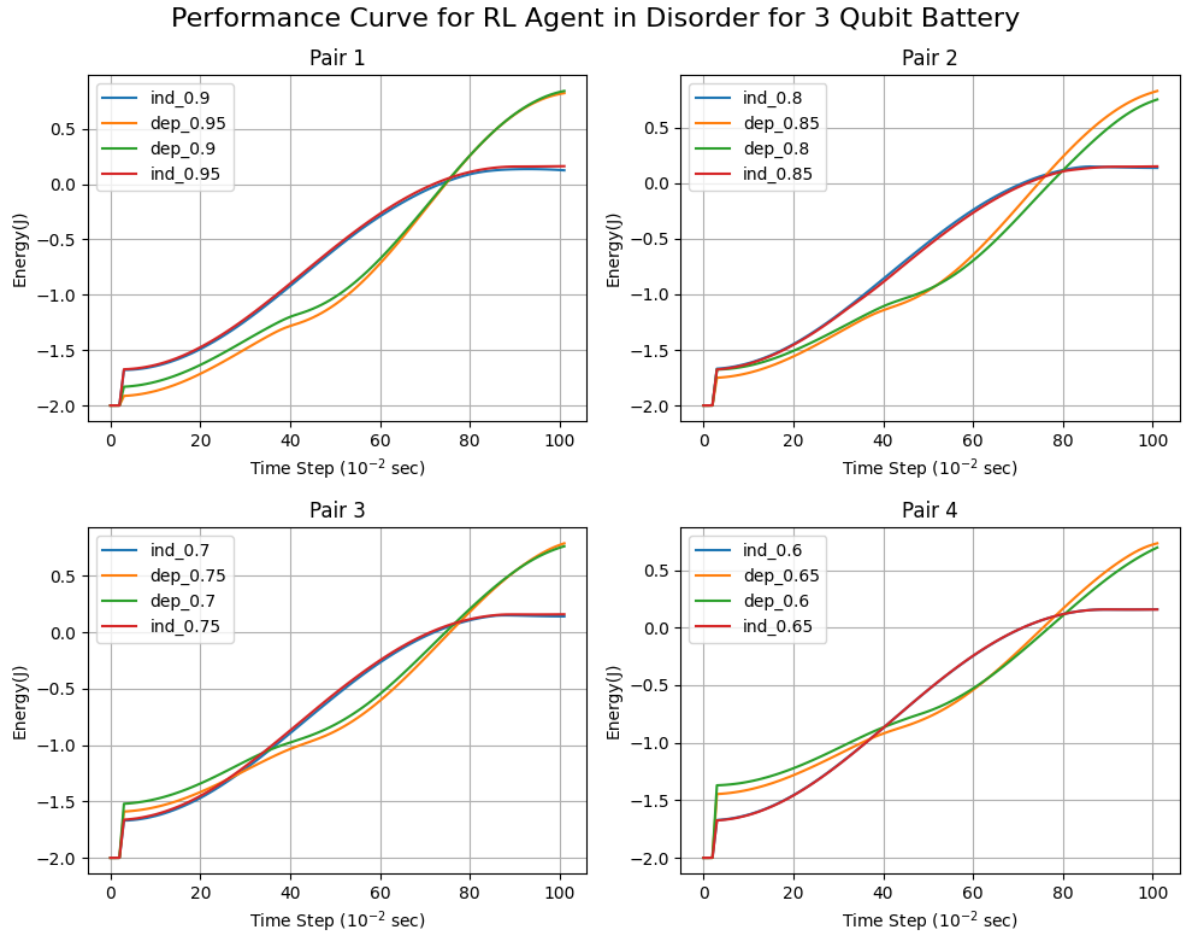


Figure 5.7: Performance of the model in disorder for a 3 qubit quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

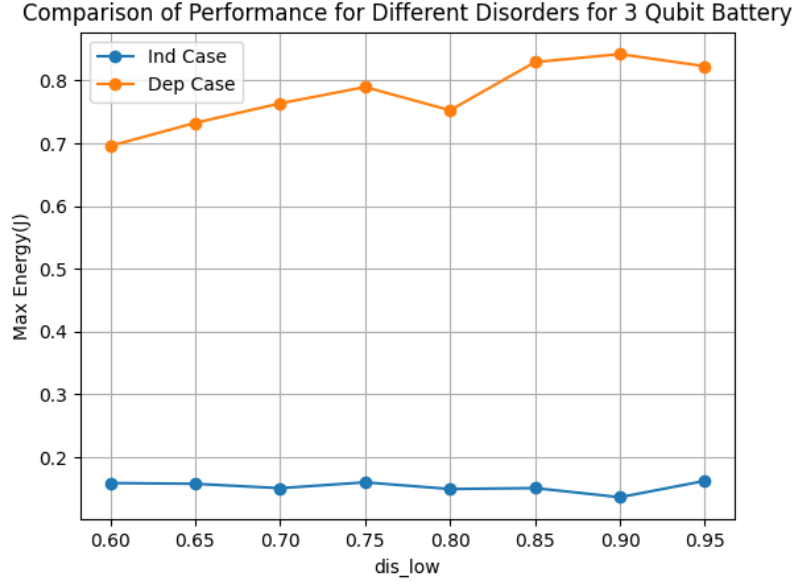


Figure 5.8: Maximum energy obtained by the model in disorder for a 3-quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

$$H_0 = J_{1,2}\sigma_1^x \otimes \sigma_2^x + J_{2,3}\sigma_2^x \otimes \sigma_3^x + J_{3,4}\sigma_3^x \otimes \sigma_4^x$$

$$H_{\text{comb}} = h'_1\sigma_1^z + h'_2\sigma_2^z + h'_3\sigma_3^z + h'_4\sigma_4^z$$

For the given Hamiltonian, we have the highest and lowest eigenvalues and their corresponding eigenvectors as:

$$e_{\max} = 3, |\psi_1\rangle \in \{|\uparrow\uparrow\uparrow\uparrow\rangle, |\downarrow\downarrow\downarrow\downarrow\rangle\}$$

$$e_{\min} = -3, |\psi_2\rangle \in \{|\downarrow\uparrow\downarrow\uparrow\rangle, |\uparrow\downarrow\uparrow\downarrow\rangle\}$$

The $|\psi_1\rangle$ and $|\psi_2\rangle$ are the possible configurations for the excited state and for the ground state of the system. The value of l according to the definition can be obtained as $l = 6$. Also, the value for the $C(t)$ for some t can be defined as

$$C(t) = \frac{E(t) + 3}{6} \times 100\%$$

5.5.1 Case 1:

We set $J_{i,i+1} = 1, \forall i \in \{1, 2, 3\}$, thus the Hamiltonian of the system can be represented as:

$$H_0 = \sigma_1^x \otimes \sigma_2^x + \sigma_2^x \otimes \sigma_3^x + \sigma_3^x \otimes \sigma_4^x$$

Since we have degeneracy, the initial state will be chosen randomly among the obtained eigenvectors corresponding to the minimum eigenvalue ($e_{\min}$), i.e., $|\psi_{\text{initial}}\rangle = |\psi_2\rangle$. The model trained for the site-dependent case performs really well and achieves and maintains $C_{\text{model}}(\tau) \geq C_{\text{greedy}}(\tau)$ for all $\tau > 1$, for $\tau = 1$ even though site dependent learn better and attain better charge percentage, site independent attain a lesser value than corresponding greedy protocol. It is also noticed that $C_{\text{SID}}(\tau = 10) = 66\%$ while the greedy protocol achieves 61%. Similarly, $C_{\text{SD}}(\tau = 10) = 98\%$, which shows the advantage of the model over the greedy protocol which achieves 95.17% as illustrated by Figure 5.10. These detailed results are mentioned in Tables 5.7 and 5.8.

Table 5.7: Performance comparison of both the models with the greedy protocol. Here SD represents the site-dependent case, SID represents the site-independent case

τ	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	0.04	-0.42	0.37	-0.45
2	1.49	0.57	2.68	0.73
4	2.18	0.6	2.75	0.92
6	2.69	0.64	2.71	0.96
8	2.7	0.67	2.87	0.94
10	2.71	0.69	2.88	0.96

5.5.2 Case 2:

We set the value $J_{i,j}$ to vary in each episode. We can see that the disordered case is well handled by the model. The trend is irregular in this case as for the site-independent case the model performs in the reverse order, i.e., the performance is higher for higher disorder which points to the need for fine-tuning of the

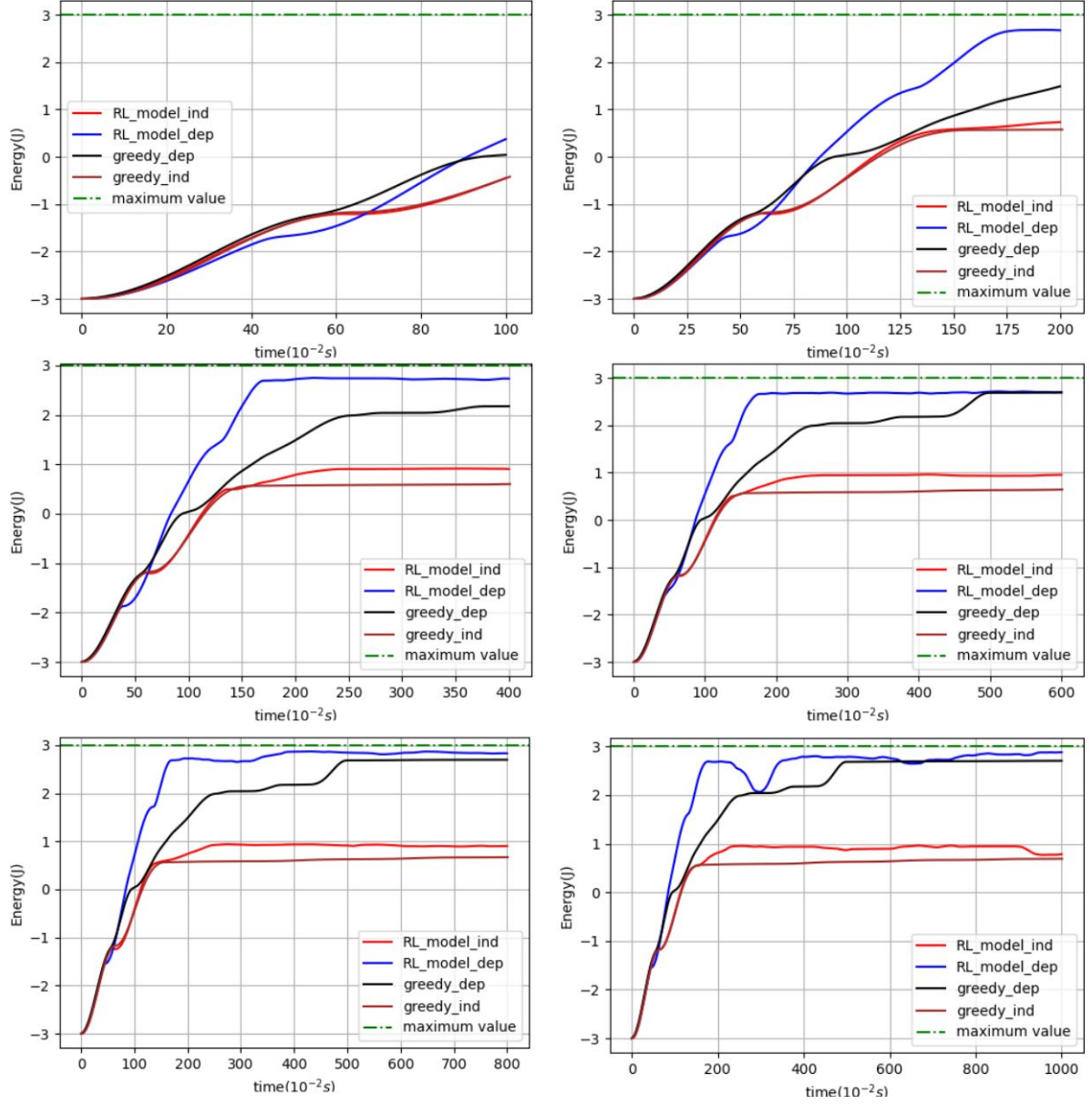


Figure 5.9: Performance of the model in comparison with the greedy protocol for a 4 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$

Table 5.8: Charge(in percentage) obtained $C(t_i)$ for each protocol in Table 5.7

$\tau(sec)$	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	50.67	43.0	56.17	42.5
2	74.83	59.5	94.67	62.17
4	86.33	60.0	95.83	65.33
6	94.83	60.67	95.17	66.0
8	95.0	61.17	97.83	65.67
10	95.17	61.5	98.0	66.0

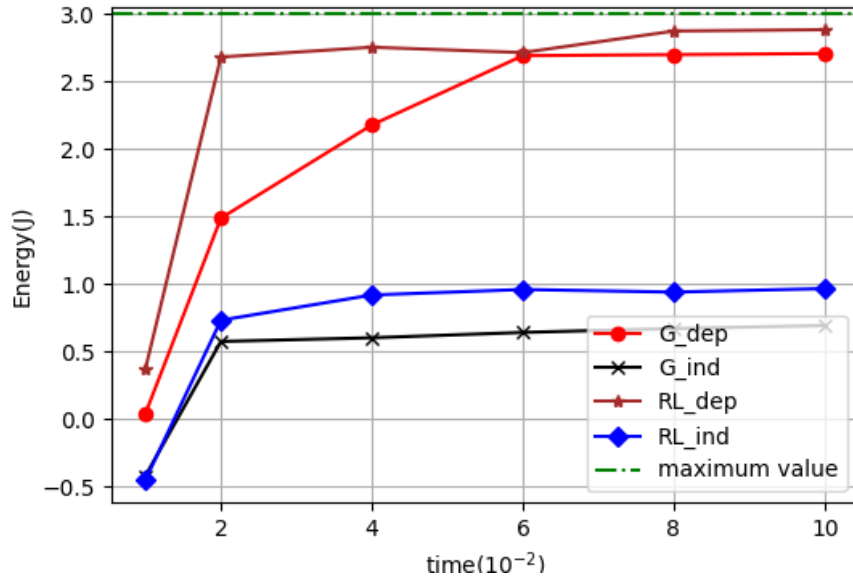


Figure 5.10: Comparison of energy obtained by the 4 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$, here the x-axis is in seconds

actor-critic network. For the site-dependent case, the trend is followed with minor irregularities as illustrated by Figure 5.12. The model's performance for the disorder is close to the normal case when $\delta = 0.95$ for both site-independent and dependent configuration5.95.8.

5.6 5-Qubit system

In this section, we will deal with a 5 qubit quantum battery with the Hamiltonian of the system as H_0 and the charging field along with the action of $\alpha(t)$ as H_{comb}

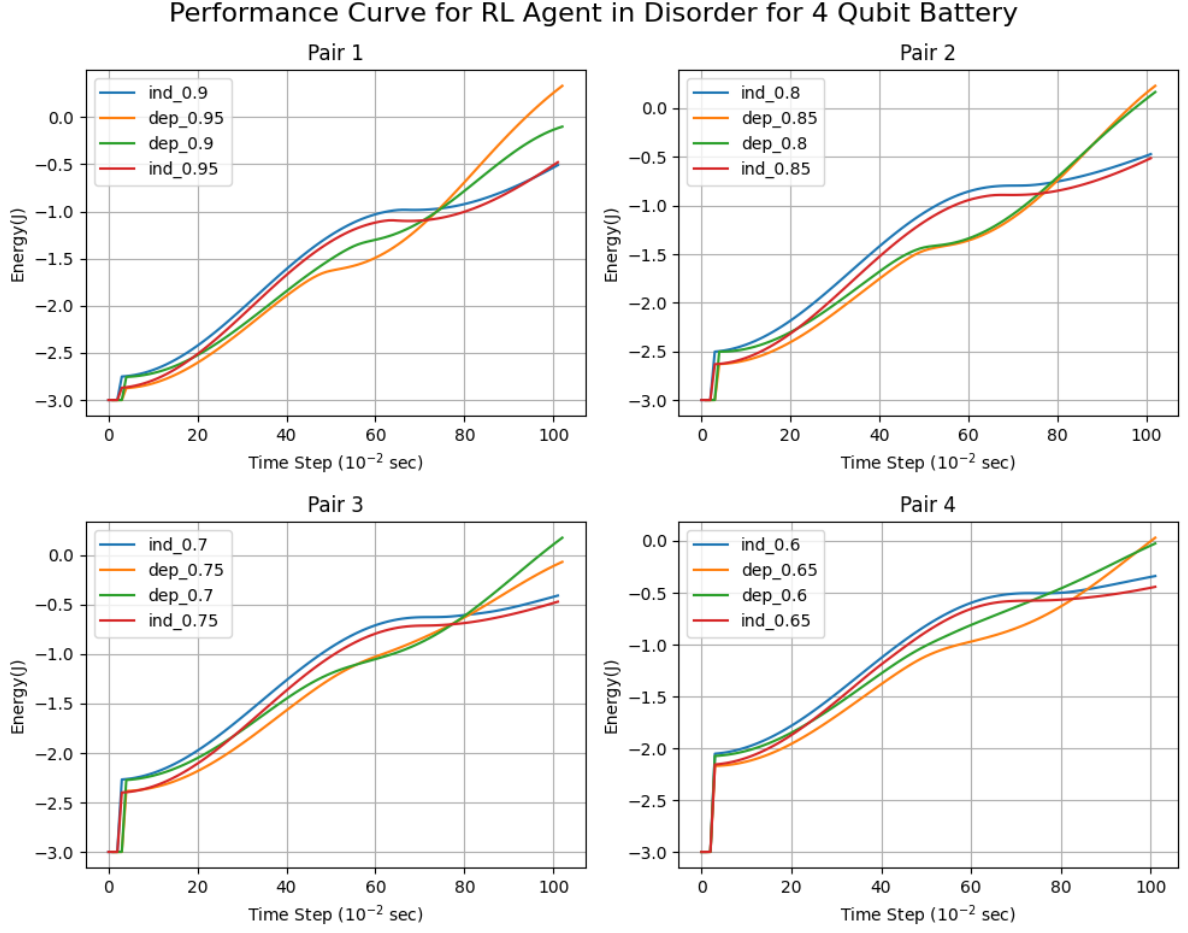


Figure 5.11: Performance of the model in disorder 4 qubit quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

Table 5.9: Table Title

δ	SID	$C_{SID}(\tau)$	SD	$C_{SD}(\tau)$
0.95	-0.479	42.02	0.3309	55.52
0.90	-0.5095	41.51	-0.1016	48.31
0.85	-0.5133	41.45	0.2304	53.84
0.80	-0.4714	42.14	0.167	52.78
0.75	-0.4706	42.16	-0.0679	48.87
0.70	-0.4087	43.19	0.1762	52.94
0.65	-0.4432	42.61	0.0304	50.51
0.60	-0.339	44.35	-0.0246	49.59

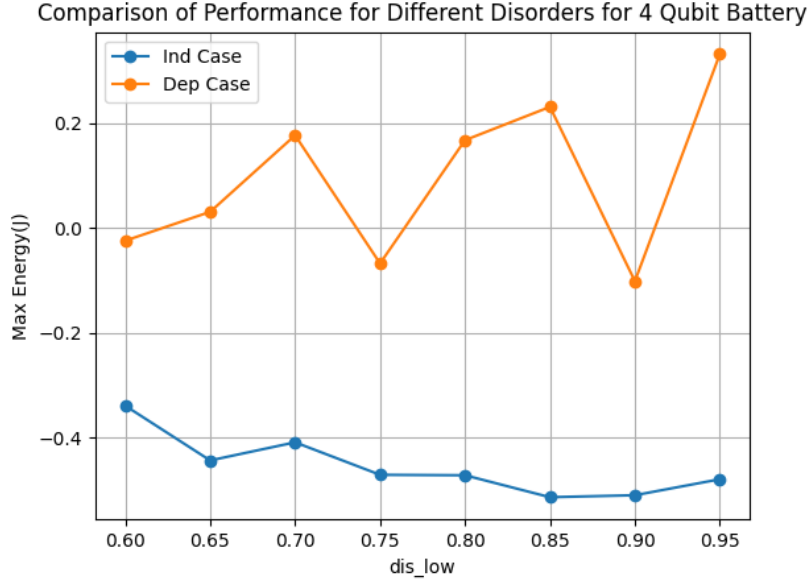


Figure 5.12: Maximum energy obtained by the model in disorder for a 4-quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

defined as:

$$H_0 = J_{1,2}\sigma_1^x \otimes \sigma_2^x + J_{2,3}\sigma_2^x \otimes \sigma_3^x + J_{3,4}\sigma_3^x \otimes \sigma_4^x + J_{4,5}\sigma_4^x \otimes \sigma_5^x$$

$$H_{\text{comb}} = h'_1\sigma_1^z + h'_2\sigma_2^z + h'_3\sigma_3^z + h'_4\sigma_4^z + h'_5\sigma_5^z$$

For the given Hamiltonian, we have the highest and lowest eigenvalues and their corresponding eigenvectors as:

$$e_{\max} = 4, |\psi_1\rangle \in \{|\uparrow\uparrow\uparrow\uparrow\rangle, |\downarrow\downarrow\downarrow\downarrow\rangle\}$$

$$e_{\min} = -4, |\psi_2\rangle \in \{|\uparrow\downarrow\uparrow\downarrow\rangle, |\downarrow\uparrow\downarrow\uparrow\rangle\}$$

The $|\psi_1\rangle$ and $|\psi_2\rangle$ are the possible configurations for the excited state and for the ground state of the system. The value of l according to the definition can be obtained as $l = 8$. Also, the value for $C(t)$ for some t can be defined as

$$C(t) = \frac{E(t) + 1}{8} \times 100\%$$

5.6.1 Case 1:

We set $J_{i,i+1} = 1, \forall i \in \{1, 2, 3, 4\}$, thus the Hamiltonian of the system can be represented as:

$$H_0 = \sigma_1^x \otimes \sigma_2^x + \sigma_2^x \otimes \sigma_3^x + \sigma_3^x \otimes \sigma_4^x + \sigma_4^x \otimes \sigma_5^x$$

Since we have degeneracy, the initial state will be chosen randomly among the obtained eigenvectors corresponding to the minimum eigenvalue ($e_{\min}$). For $\tau = 1$, the site-independent model achieves better results than the greedy protocol for the corresponding configuration, i.e., $C_{SID}(\tau = 1) > C_{greedy}(\tau = 1)$. However, as τ increases, the model performs better compared to the greedy protocol. For $\tau = 10$, the site-dependent model achieves a value of 95% while the greedy protocol for the same configuration achieves 91%. Similarly, the site-independent model achieves 85.88%, while the greedy protocol attains 79%. These results demonstrate the beneficial role of reinforcement learning in finding protocols (5.11). Detailed results are illustrated by the curves in Figures 5.13 and 5.14.

Table 5.10: Performance comparison of both the models with the greedy protocol. Here SD represents the site-dependent case, SID represents the site-independent case

$\tau(sec)$	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	0.2	-0.39	-0.0	-0.18
2	2.49	1.0	2.88	1.01
4	3.25	1.01	3.33	1.15
6	3.27	1.64	3.56	2.49
8	3.28	1.82	3.57	2.58
10	3.29	2.32	3.64	2.87

5.6.2 case 2:

The general trend of attaining better results for lesser disorder (i.e., for higher values of δ) is followed with minor exceptions, as illustrated by Figures 5.16

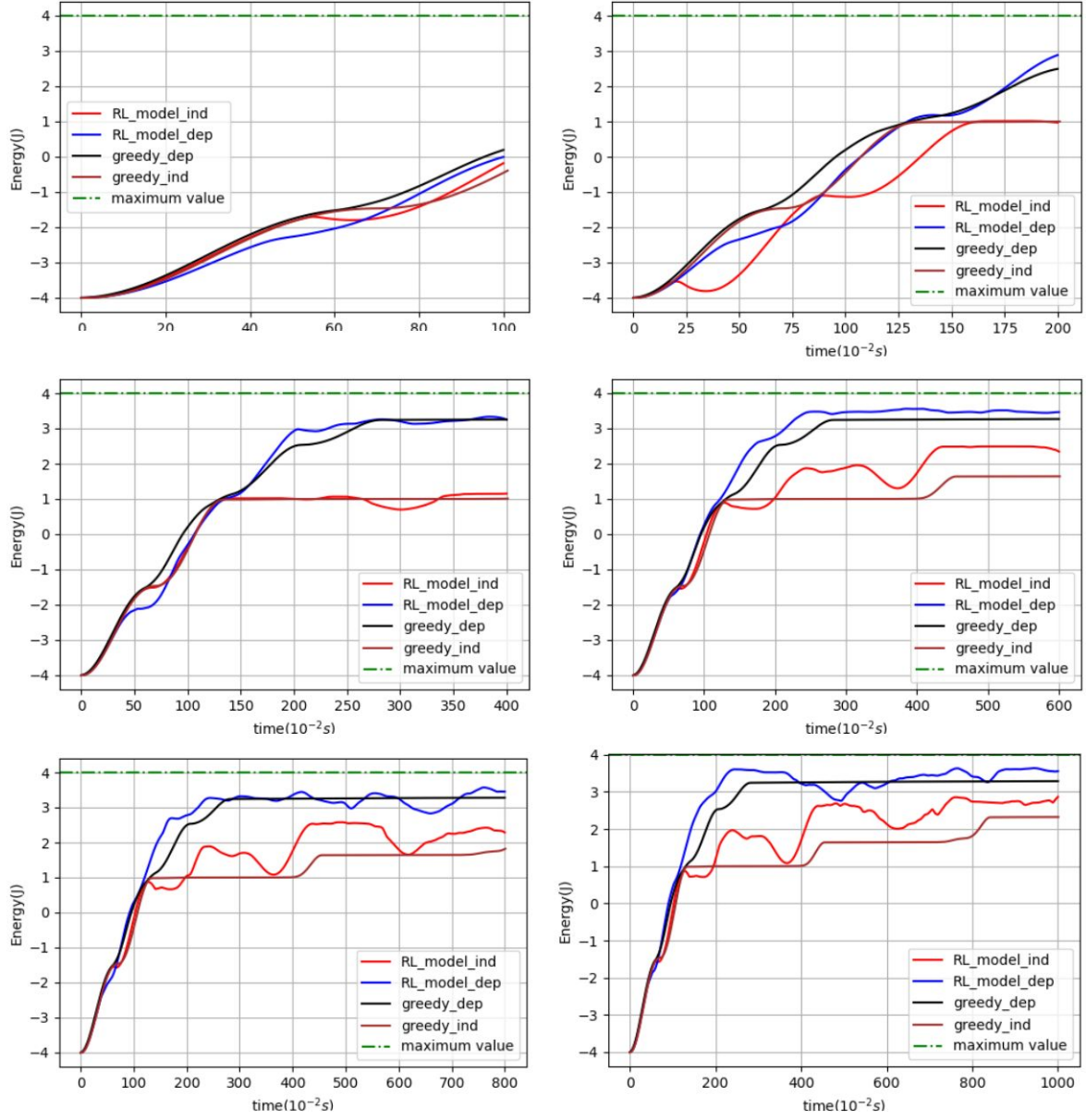


Figure 5.13: Performance of the model in comparison with the greedy protocol for a 5 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$

Table 5.11: Charge(in percentage) obtained $C(t_i)$ for each protocol in Table 5.10

$\tau(sec)$	Greedy-SD	Greedy-SID	Model-SD	Model-SID
1	52.5	45.12	50.0	47.75
2	81.12	62.5	86.0	62.62
4	90.62	62.62	91.62	64.38
6	90.88	70.5	94.5	81.12
8	91.0	72.75	94.62	82.25
10	91.12	79.0	95.5	85.88

and 5.15. The site-dependent case attains a higher percentage when compared to the site-independent case. For both site-dependent and site-independent cases, the model's performance in disorder is comparable to the model's performance in the normal case. This can be seen from Tables 5.11 and 5.12.

Table 5.12: Table Title

δ	SID	$C_{SID}(\tau)$	SD	$C_{SD}(\tau)$
0.95	-0.2923	46.35	-0.0125	49.84
0.90	-0.3539	45.58	-0.0427	49.47
0.85	-0.5456	43.18	-0.0745	49.07
0.80	-0.5356	43.31	-0.1628	47.96
0.75	-0.5006	43.74	-0.0867	48.92
0.70	-0.5109	43.61	-0.2517	46.85
0.65	-0.4727	44.09	-0.3682	45.40
0.60	-0.4425	44.47	-0.0711	49.11

5.7 conclusion:

In this work, we have demonstrated the significant impact of utilizing machine learning techniques on the advancement of quantum technologies. we tried to address the challenge of finding optimal control, especially in the presence of disorders. We addressed this challenge through the application of reinforcement learning, using the information of system's dynamics described by equations

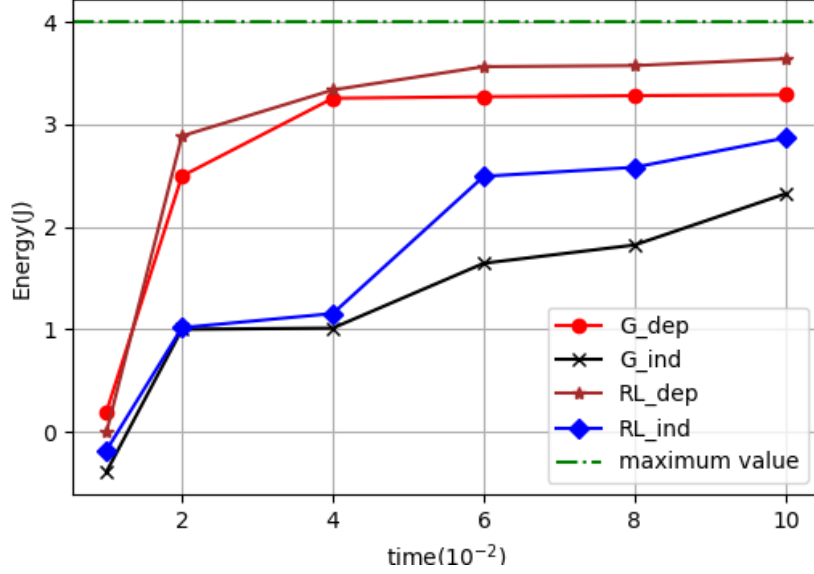


Figure 5.14: Comparison of energy obtained by the 5 qubit quantum battery for $\tau \in \{1, 2, 4, 6, 8, 10\}$, here the x-axis is in seconds

governing the system. By formulating it as a reinforcement learning problem, we derived a policy that aids in finding the optimal control for a given system.

Our investigation began with a single qubit quantum battery and was subsequently extended to multi-qubit batteries. We obtained results for batteries consisting of 2, 3, 4, and 5 qubits across different time intervals, including 1, 2, 4, 6, 8, and 10 seconds. Due to computational constraints, our study was limited to 5 qubits, as the training time became difficult to accommodate computationally. Despite computational challenges, we achieved promising results, particularly in scenarios involving disorder. However, due to computational constraints, we restricted our investigation to a time interval of 1 second.

On behalf of our work, we demonstrated that the use of machine learning can assist us in both theoretical and experimental fields, thereby aiding researchers in quantum technology to gain better momentum. We would like to emphasize reinforcement learning as an important part of this advancement. Since the concept of using neural networks to approximate functions is already in action, one can model the system using available data and employ reinforcement learning to find the optimal strategy for experiments, leading to improved results.

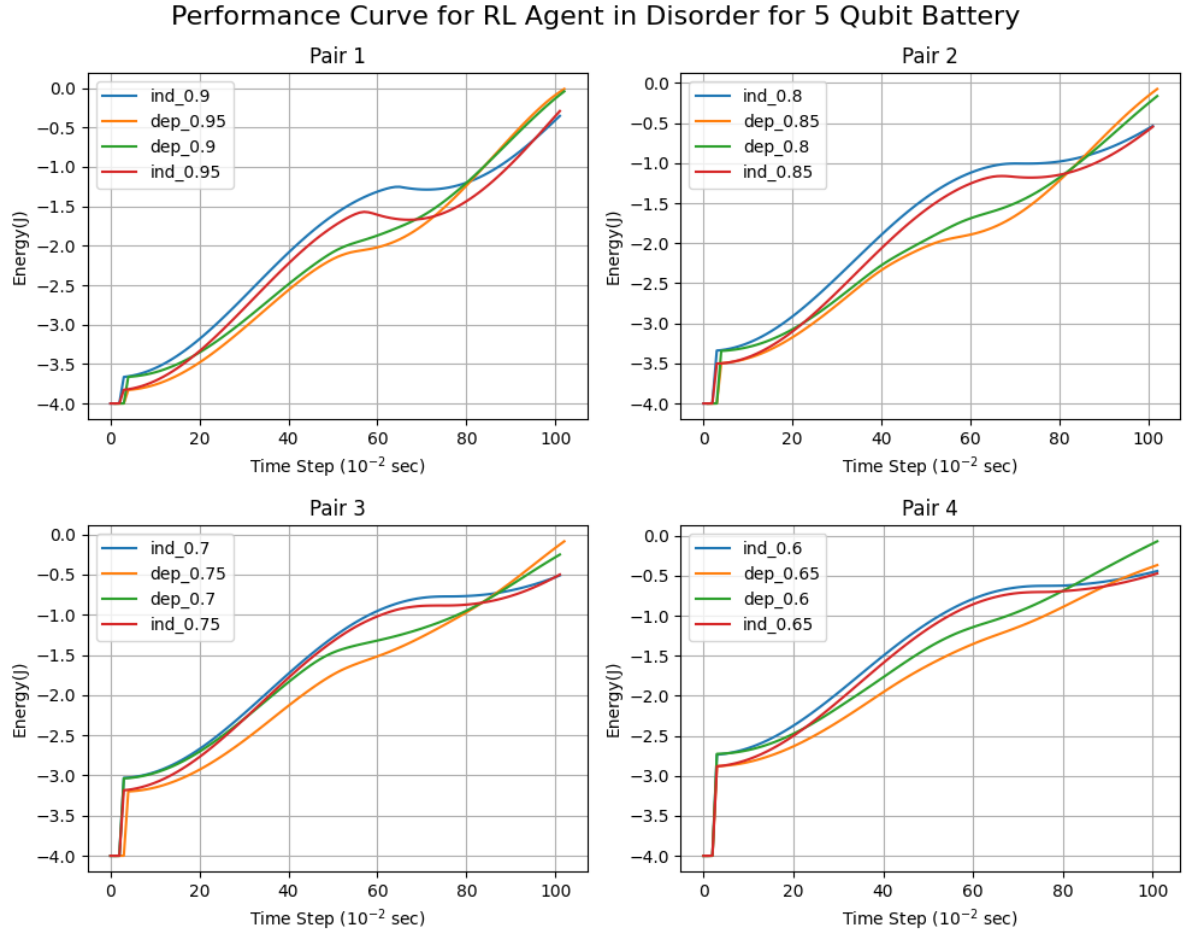


Figure 5.15: Performance of the model in disorder for a 5-quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

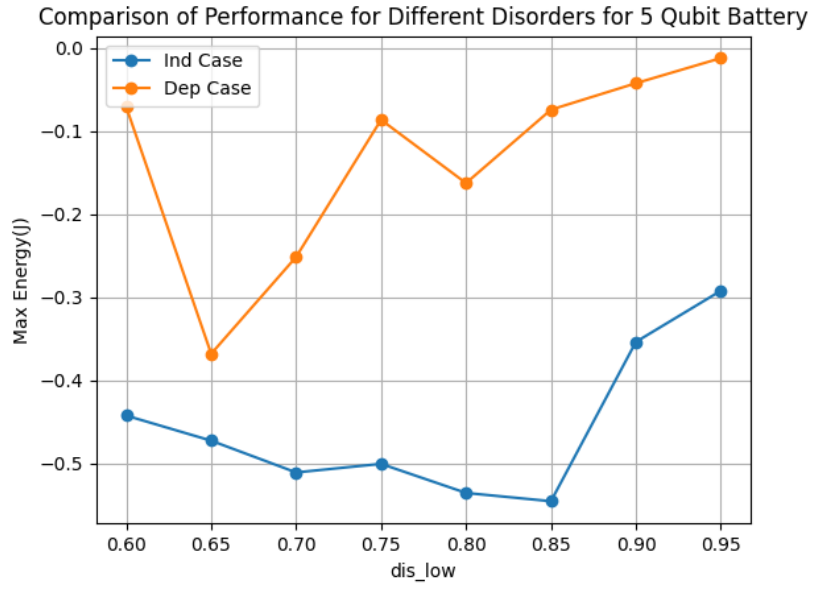


Figure 5.16: Maximum energy obtained by the model in disorder for a 5 qubit quantum battery for $\delta \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$

Bibliography

- [1] C S Adams, J D Pritchard, and J P Shaffer. Rydberg atom quantum technologies. *Journal of Physics B: Atomic, Molecular and Optical Physics*, 53(1):012002, December 2019.
- [2] Cesar O. Aguilar and Arthur J. Krener. Numerical solutions to the bellman equation of optimal control. *Journal of Optimization Theory and Applications*, 160(2):527–552, September 2013.
- [3] Taha Alfakih, Mohammad Mehedi Hassan, Abdu Gumaei, Claudio Savaglio, and Giancarlo Fortino. Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on sarsa. *IEEE Access*, 8:54074–54084, 2020.
- [4] Robert Alicki and Mark Fannes. Entanglement boost for extractable work from ensembles of quantum batteries. *Phys. Rev. E*, 87:042123, Apr 2013.
- [5] Robert Alicki and Mark Fannes. Entanglement boost for extractable work from ensembles of quantum batteries. *Phys. Rev. E*, 87:042123, Apr 2013.
- [6] Gian Marcello Andolina, Donato Farina, Andrea Mari, Vittorio Pellegrini, Vittorio Giovannetti, and Marco Polini. Charger-mediated energy transfer in exactly solvable models for quantum batteries. *Phys. Rev. B*, 98:205423, Nov 2018.
- [7] Gian Marcello Andolina, Donato Farina, Andrea Mari, Vittorio Pellegrini, Vittorio Giovannetti, and Marco Polini. Charger-mediated energy transfer

- in exactly solvable models for quantum batteries. *Phys. Rev. B*, 98:205423, Nov 2018.
- [8] Gian Marcello Andolina, Maximilian Keck, Andrea Mari, Michele Campisi, Vittorio Giovannetti, and Marco Polini. Extractable work, the role of correlations, and asymptotic freedom in quantum batteries. *Phys. Rev. Lett.*, 122:047702, Feb 2019.
 - [9] John F. Barry, Jennifer M. Schloss, Erik Bauch, Matthew J. Turner, Connor A. Hart, Linh M. Pham, and Ronald L. Walsworth. Sensitivity optimization for nv-diamond magnetometry. *Rev. Mod. Phys.*, 92:015004, Mar 2020.
 - [10] C. H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.
 - [11] Felix C Binder, Sai Vinjanampathy, Kavan Modi, and John Goold. Quantacell: powerful charging of quantum batteries. *New Journal of Physics*, 17(7):075015, July 2015.
 - [12] Alexandre Blais, Arne L. Grimsmo, S. M. Girvin, and Andreas Wallraff. Circuit quantum electrodynamics. *Rev. Mod. Phys.*, 93:025005, May 2021.
 - [13] Colin D. Bruzewicz, John Chiaverini, Robert McConnell, and Jeremy M. Sage. Trapped-ion quantum computing: Progress and challenges. *Applied Physics Reviews*, 6(2), may 2019.
 - [14] Guido Burkard, Thaddeus D. Ladd, Andrew Pan, John M. Nichol, and Jason R. Petta. Semiconductor spin qubits. *Reviews of Modern Physics*, 95(2), June 2023.
 - [15] Francesco Campaioli, Stefano Gherardini, James Q. Quach, Marco Polini, and Gian Marcello Andolina. Colloquium: Quantum batteries, 2023.

- [16] Francesco Campaioli, Felix A. Pollock, Felix C. Binder, Lucas Céleri, John Goold, Sai Vinjanampathy, and Kavan Modi. Enhancing the charging power of quantum batteries. *Phys. Rev. Lett.*, 118:150601, Apr 2017.
- [17] Francesco Campaioli, Felix A. Pollock, Felix C. Binder, Lucas Céleri, John Goold, Sai Vinjanampathy, and Kavan Modi. Enhancing the charging power of quantum batteries. *Phys. Rev. Lett.*, 118:150601, Apr 2017.
- [18] Francesco Casola, Toeno van der Sar, and Amir Yacoby. Probing condensed matter physics with magnetometry based on nitrogen-vacancy centres in diamond. *Nature Reviews Materials*, 3(1), January 2018.
- [19] Michael Castronovo, Francis Maes, Raphael Fonteneau, and Damien Ernst. Learning exploration/exploitation strategies for single trajectory reinforcement learning. In Marc Peter Deisenroth, Csaba Szepesvári, and Jan Peters, editors, *Proceedings of the Tenth European Workshop on Reinforcement Learning*, volume 24 of *Proceedings of Machine Learning Research*, pages 1–10, Edinburgh, Scotland, 30 Jun–01 Jul 2013. PMLR.
- [20] Jesse Clifton and Eric Laber. Q-learning: Theory and applications. *Annual Review of Statistics and Its Application*, 7(1):279–301, March 2020.
- [21] Kalyan Das, Jiming Jiang, and J. N. K. Rao. Mean squared error of empirical predictor. *The Annals of Statistics*, 32(2), April 2004.
- [22] Sebastian Deffner and Eric Lutz. Energy–time uncertainty relation for driven quantum systems. *Journal of Physics A: Mathematical and Theoretical*, 46(33):335302, July 2013.
- [23] R. H. Dicke. Coherence in spontaneous radiation processes. *Phys. Rev.*, 93:99–110, Jan 1954.
- [24] Jianqing Fan, Zhaoran Wang, Yuchen Xie, and Zhuoran Yang. A theoretical analysis of deep q-learning. In Alexandre M. Bayen, Ali Jadbabaie, George Pappas, Pablo A. Parrilo, Benjamin Recht, Claire Tomlin, and

- Melanie Zeilinger, editors, *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pages 486–489. PMLR, 10–11 Jun 2020.
- [25] Donato Farina, Gian Marcello Andolina, Andrea Mari, Marco Polini, and Vittorio Giovannetti. Charger-mediated energy transfer for quantum batteries: An open-system approach. *Phys. Rev. B*, 99:035421, Jan 2019.
 - [26] Donato Farina, Gian Marcello Andolina, Andrea Mari, Marco Polini, and Vittorio Giovannetti. Charger-mediated energy transfer for quantum batteries: An open-system approach. *Phys. Rev. B*, 99:035421, Jan 2019.
 - [27] Dario Ferraro, Michele Campisi, Gian Marcello Andolina, Vittorio Pellegrini, and Marco Polini. High-power collective charging of a solid-state quantum battery. *Physical Review Letters*, 120(11), Mar 2018.
 - [28] Dario Ferraro, Michele Campisi, Gian Marcello Andolina, Vittorio Pellegrini, and Marco Polini. High-power collective charging of a solid-state quantum battery. *Phys. Rev. Lett.*, 120:117702, Mar 2018.
 - [29] Dario Ferraro, Michele Campisi, Gian Marcello Andolina, Vittorio Pellegrini, and Marco Polini. High-power collective charging of a solid-state quantum battery. *Phys. Rev. Lett.*, 120:117702, Mar 2018.
 - [30] Dario Ferraro, Michele Campisi, Gian Marcello Andolina, Vittorio Pellegrini, and Marco Polini. High-power collective charging of a solid-state quantum battery. *Phys. Rev. Lett.*, 120:117702, Mar 2018.
 - [31] Nicolai Friis and Marcus Huber. Precision and work fluctuations in gaussian battery charging, 2018.
 - [32] R. Gentleman and V. J. Carey. *Unsupervised Machine Learning*, page 137–157. Springer New York, 2008.
 - [33] Dibya Ghosh and Marc G. Bellemare. Representations for stable off-policy reinforcement learning. In Hal Daumé III and Aarti Singh, editors,

Proceedings of the 37th International Conference on Machine Learning, volume 119 of *Proceedings of Machine Learning Research*, pages 3556–3565. PMLR, 13–18 Jul 2020.

- [34] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. Quantum limits to dynamical evolution. *Phys. Rev. A*, 67:052109, May 2003.
- [35] Alessandro Giuseppe and Antonio Pietrabissa. Bellman’s principle of optimality and deep reinforcement learning for time-varying tasks. *International Journal of Control*, 95(9):2448–2459, April 2021.
- [36] Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning, 2019.
- [37] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.
- [38] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications, 2019.
- [39] Jeff Heaton. Ian goodfellow, yoshua bengio, and aaron courville: Deep learning: The mit press, 2016, 800 pp, isbn: 0262035618. *Genetic Programming and Evolvable Machines*, 19(1–2):305–307, October 2017.
- [40] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, Gabriel Dulac-Arnold, John Agapiou, Joel Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), April 2018.

- [41] Karen V. Hovhannisyan, Martí Perarnau-Llobet, Marcus Huber, and Antonio Acín. Entanglement generation is not necessary for optimal work extraction. *Phys. Rev. Lett.*, 111:240401, Dec 2013.
- [42] Shin Ishii, Wako Yoshida, and Junichiro Yoshimoto. Control of exploitation–exploration meta-parameter in reinforcement learning. *Neural Networks*, 15(4–6):665–687, June 2002.
- [43] J Jaramillo, M Beau, and A del Campo. Quantum supremacy of many-particle thermal machines. *New Journal of Physics*, 18(7):075019, Jul 2016.
- [44] J.R. Johansson, P.D. Nation, and Franco Nori. Qutip: An open-source python framework for the dynamics of open quantum systems. *Computer Physics Communications*, 183(8):1760–1772, August 2012.
- [45] Thao P. Le, Jesper Levinsen, Kavan Modi, Meera M. Parish, and Felix A. Pollock. Spin-chain model of a many-body quantum battery. *Physical Review A*, 97(2), Feb 2018.
- [46] Thao P. Le, Jesper Levinsen, Kavan Modi, Meera M. Parish, and Felix A. Pollock. Spin-chain model of a many-body quantum battery. *Phys. Rev. A*, 97:022106, Feb 2018.
- [47] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, May 2015.
- [48] Felix Leibfried, Sergio Pascual-Díaz, and Jordi Grau-Moya. A unified bellman optimality principle combining reward maximization and empowerment. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [49] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review, 2018.

- [50] Yuxi Li. Deep reinforcement learning: An overview. *CoRR*, abs/1701.07274, 2017.
- [51] Jingbin Liu, Xinyang Gu, and Shuai Liu. Policy optimization reinforcement learning with entropy regularization, 2020.
- [52] Ruishan Liu and James Zou. The effects of memory replay in reinforcement learning. In *2018 56th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 478–485, 2018.
- [53] Francesco Mazzoncini, Vasco Cavina, Gian Marcello Andolina, Paolo Andrea Erdman, and Vittorio Giovannetti. Optimal control methods for quantum batteries. *Physical Review A*, 107(3), March 2023.
- [54] Vladimir Nasteski. An overview of the supervised machine learning methods. *HORIZONS.B*, 4:51–62, December 2017.
- [55] Rui Nian, Jinfeng Liu, and Biao Huang. A review on reinforcement learning: Introduction and applications in industrial process control. *Computers amp; Chemical Engineering*, 139:106886, August 2020.
- [56] Maximilian E. Ororbia and Gordon P. Warn. Design synthesis through a markov decision process and reinforcement learning framework. *Journal of Computing and Information Science in Engineering*, 22(2), July 2021.
- [57] Davide Rossini, Gian Marcello Andolina, and Marco Polini. Many-body localized quantum batteries. *Physical Review B*, 100(11), Sep 2019.
- [58] Davide Rossini, Gian Marcello Andolina, and Marco Polini. Many-body localized quantum batteries. *Phys. Rev. B*, 100:115142, Sep 2019.
- [59] Subir Sachdev and Jinwu Ye. Gapless spin-fluid ground state in a random quantum heisenberg magnet. *Phys. Rev. Lett.*, 70:3339–3342, May 1993.
- [60] Romana Schirhagl, Kevin Chang, Michael Loretz, and Christian L. Degen. Nitrogen-vacancy centers in diamond: Nanoscale sensors for physics and

- biology. *Annual Review of Physical Chemistry*, 65(Volume 65, 2014):83–105, 2014.
- [61] Kristina P. Sinaga and Miin-Shen Yang. Unsupervised k-means clustering algorithm. *IEEE Access*, 8:80716–80727, 2020.
 - [62] Haoran Wang, Thaleia Zariphopoulou, and Xunyu Zhou. Exploration versus exploitation in reinforcement learning: a stochastic control approach, 2019.
 - [63] Xu Wang, Sen Wang, Xingxing Liang, Dawei Zhao, Jincai Huang, Xin Xu, Bin Dai, and Qiguang Miao. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4):5064–5078, 2024.
 - [64] Y. K. Wang and F. T. Hioe. Phase transition in the dicke model of super-radiance. *Phys. Rev. A*, 7:831–836, Mar 1973.
 - [65] Yinmin Zhang, Jie Liu, Chuming Li, Yazhe Niu, Yaodong Yang, Yu Liu, and Wanli Ouyang. A perspective of q-value estimation on offline-to-online reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(15):16908–16916, March 2024.
 - [66] Dongbin Zhao, Haitao Wang, Kun Shao, and Yuanheng Zhu. Deep reinforcement learning with experience replay based on sarsa. In *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1–6, 2016.
 - [67] Rui Zhao, Xudong Sun, and Volker Tresp. Maximum entropy-regularized multi-goal reinforcement learning. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 7553–7562. PMLR, 09–15 Jun 2019.